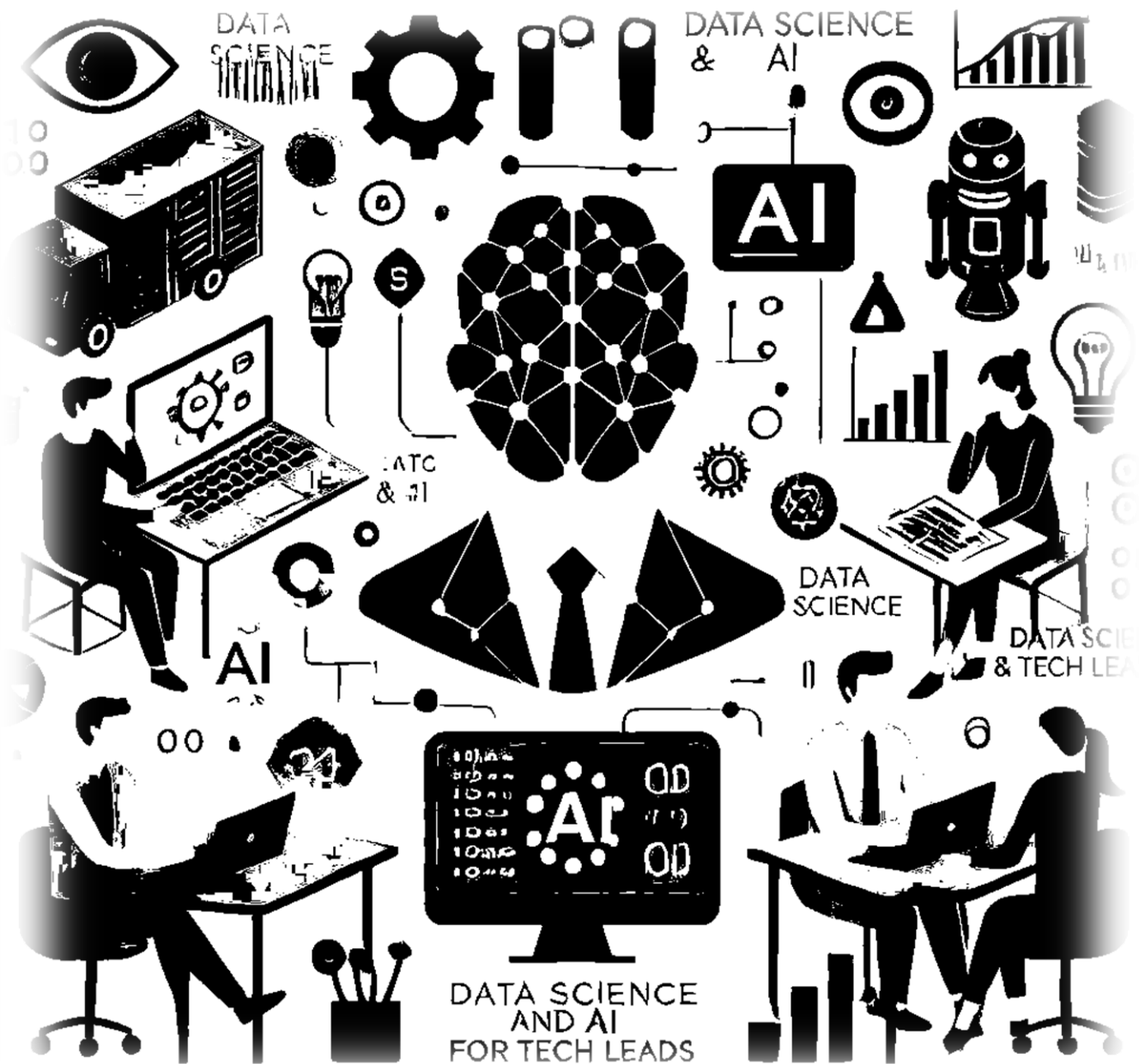


Especialização Tech Lead

Data Science & IA

2025.02

Prof. Vitor Casadei – vc@cesar.school



Agenda

1. Introdução
2. Aprendizado
3. Regressão Linear
4. Regressão Logística
5. Overfitting e Underfitting
6. Redes Neurais
7. Machine Learning e Deep Learning

Definição de Machine Learning

Machine learning: Field of study that **gives computers the ability to learn** without being explicitly programmed.

Arthur Samuel, 1959

Definição de Machine Learning

Well posed Learning Program: A computer is said to learn from experience **E** with respect to some class of task **T** and performance measure **P**, if it's performance at tasks in **T**, as measured by **P**, improves with experience **E**.

Tom Mitchell, 1998

Definição de Machine Learning

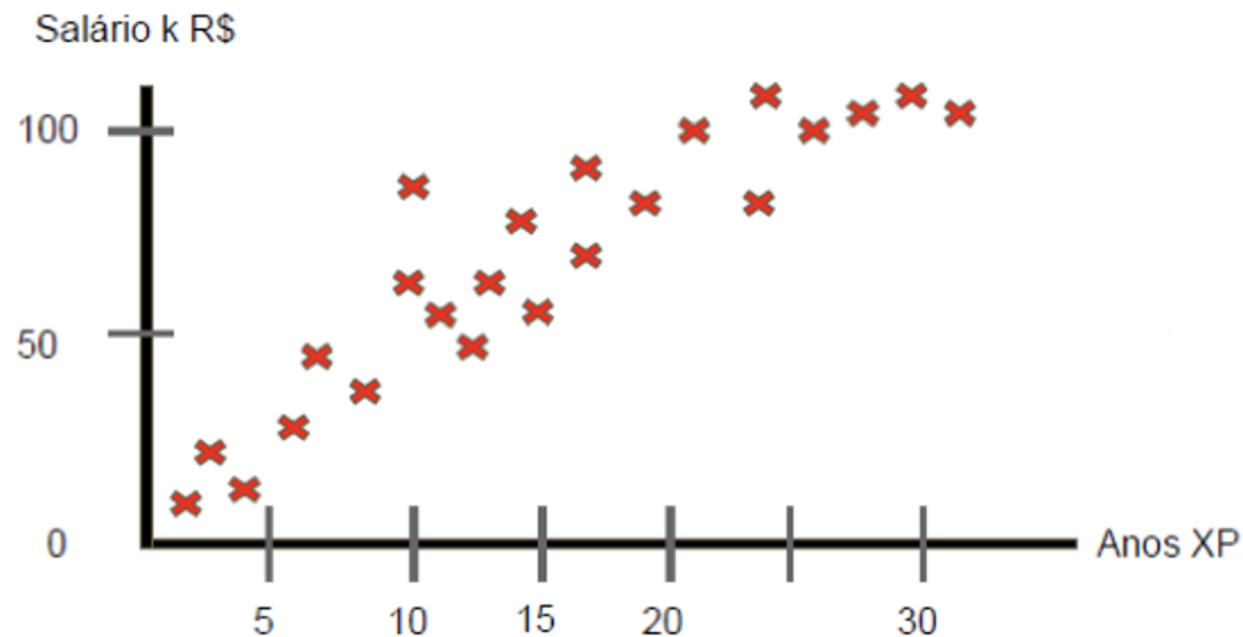
Well posed Learning Program: A computer is said to learn from experience **E** with respect to some class of task **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

Tom Mitchell, 1998

➤ Spam Classification:

- **E**: Watching you label emails as SPAM/Not SPAM
- **T**: Classifying emails as SPAM/Not SPAM
- **P**: The number (fraction) of emails correctly classified

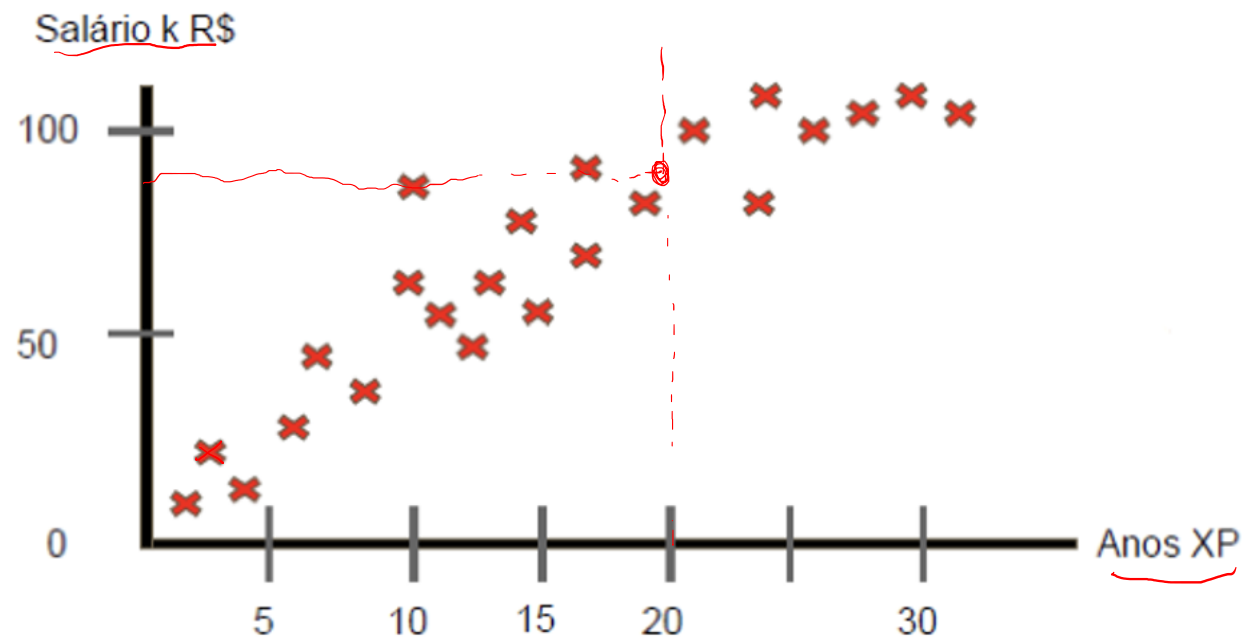
Aprendizado Supervisionado



Aprendizado com base nas
"respostas certas"

→ **Regressão:** Predição de Valores Contínuos

Aprendizado Supervisionado

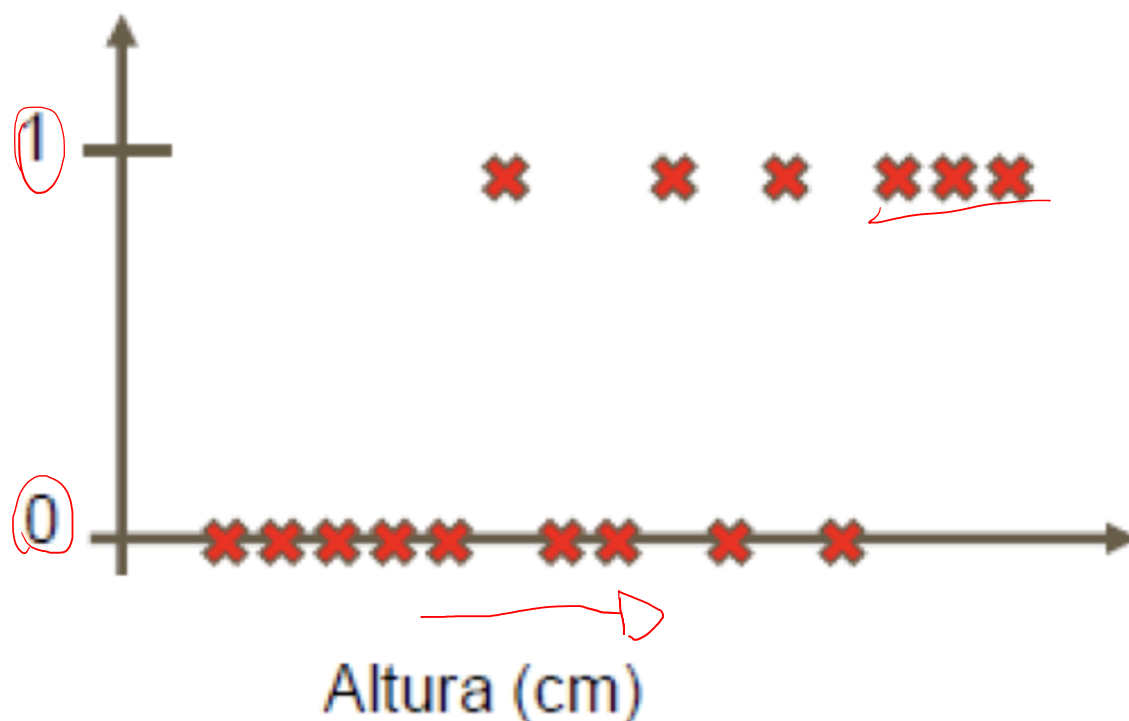


Aprendizado com base nas
"respostas certas"

Regressão: Predição de Valores Contínuos

Aprendizado Supervisionado

Pessoa Adulto

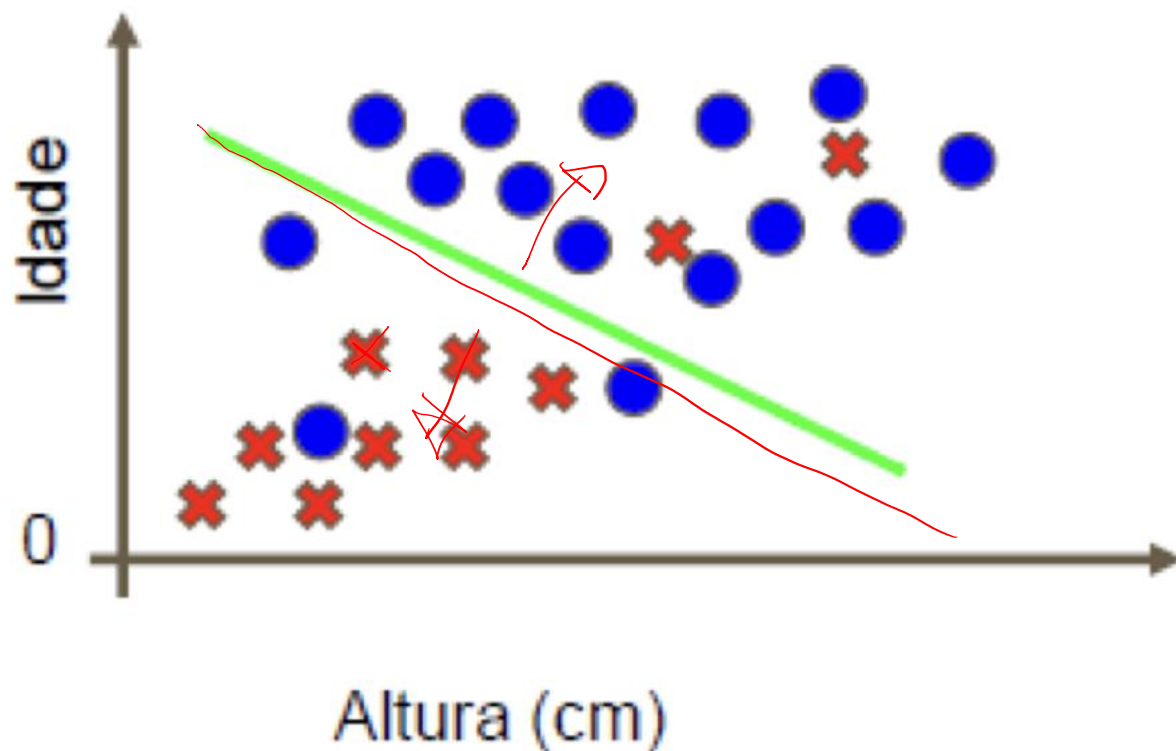


Aprendizado com base nas
"respostas certas"

Classificação: Predição de Valores
Discretos

Aprendizado Supervisionado

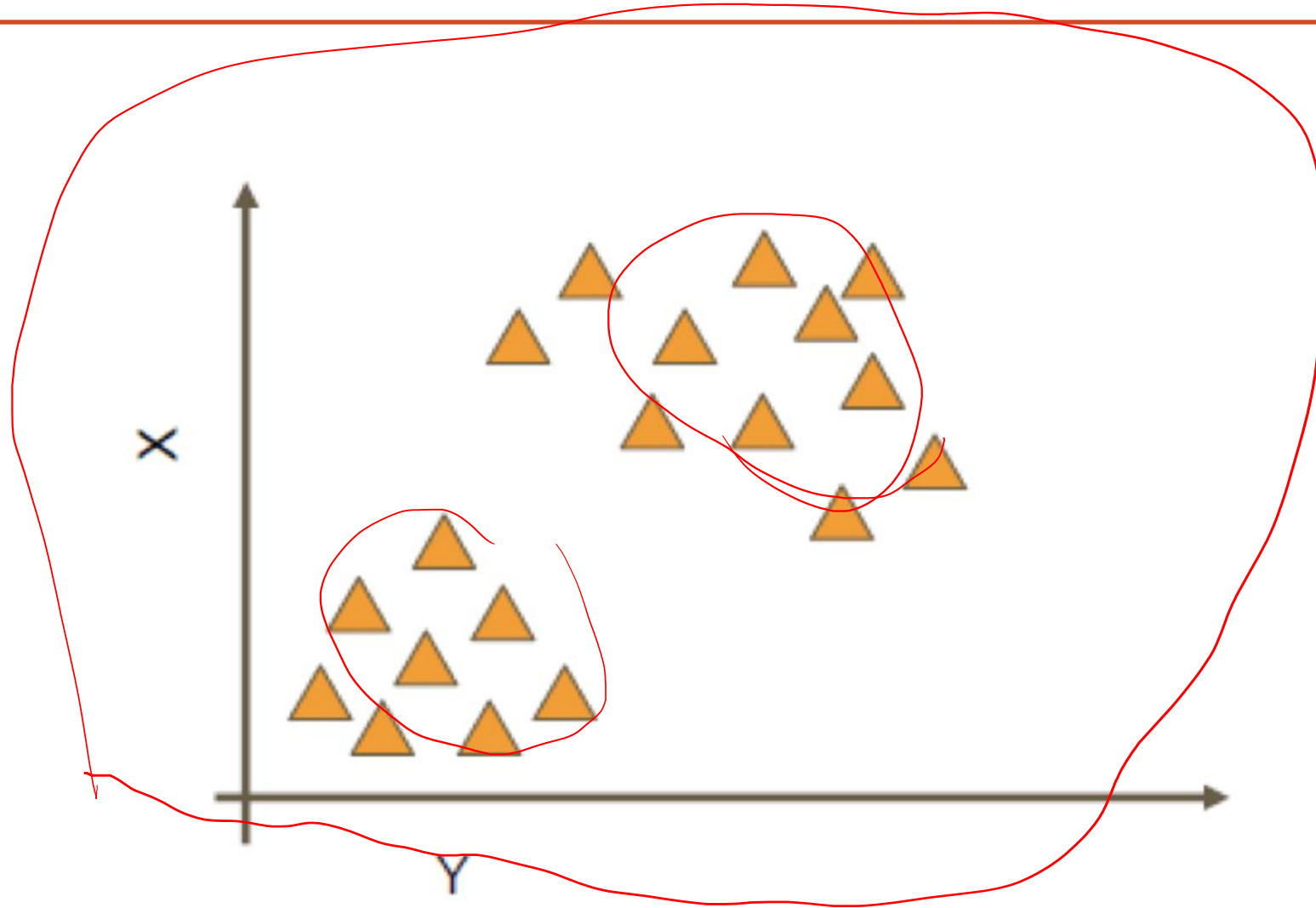
Pessoa Adulto



Aprendizado com base nas
"respostas certas"

Classificação: Predição de Valores Discretos

Aprendizado Não Supervisionado



Aprendizado Não Supervisionado

Interaction with mobile devices by elderly people: the Brazilian scenario.
Leme, R.R., Zaina, L.A. and Casadei, V., 2014



Manoel

Retired man, 70 years.

"Technology is not just for the younger"

Has a college degree, is an active member of Third Age group and well respected by people.
When friends of Mr. Manoel has doubts about new technologies, he readily help because they know he can not live without your smartphone, with which ships with the latest news.

Personal goals:

Do not waste time with technology and enjoy it the best way possible.

Practical goals:

The text of the programs on the mobile device could be a little bigger.



Marina

Retired woman, 80 years.

"I never had a cell phone!"

Although they enjoy good health, due to the limitations imposed by age, Marina depends on the family to perform basic tasks such as call to the grandchildren or watch a DVD. Regrets are just not having completed primary education. Carefree, proud to say I never laid a hand on a computer. Despite the insistence of the children, is reluctant to have a mobile device by consider it impossible to use.

Personal goals:

Enjoy life with their grandchildren. If you need the technology they can help me.

Practical goals:

The devices are too complicated to operate. I'm afraid to use them.



Lucia

A worker retired, 60 years.

"I'm retired but I can't stay without working"

Lucia is a person who although already retired, as the value of benefit is not enough, continues to work.
A few months ago bought a mobile device and is asking all friends how to use it. Regularly attend the computer classes and is finalizing the school. Is becoming a fan of games on mobile.

Personal goals:

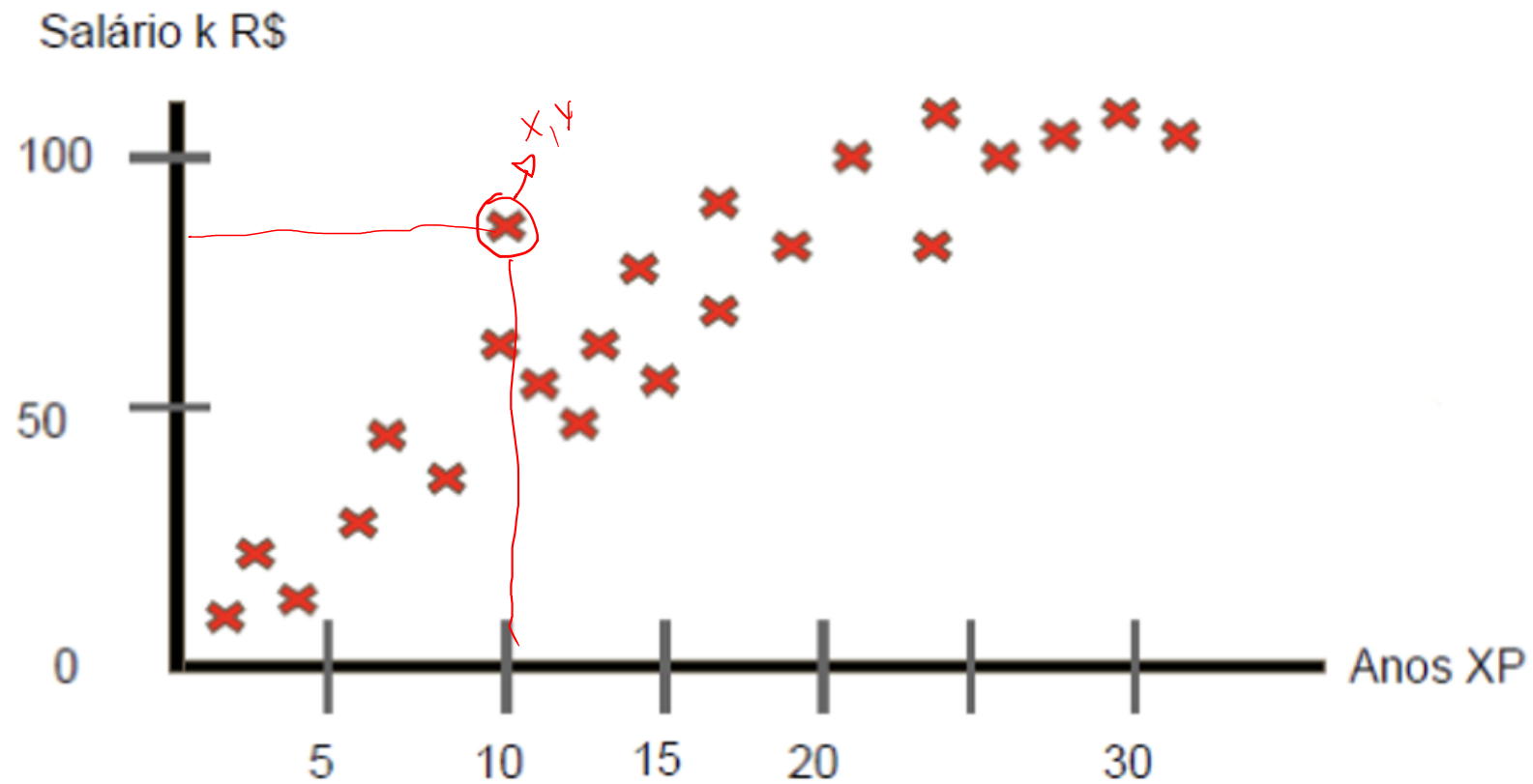
Learn more. Technology should support us constantly.

Practical goals:

At times I have trouble clicking on buttons, especially when the images are very small.

Regressão Linear – Uma variável

Predição de Salário Anual



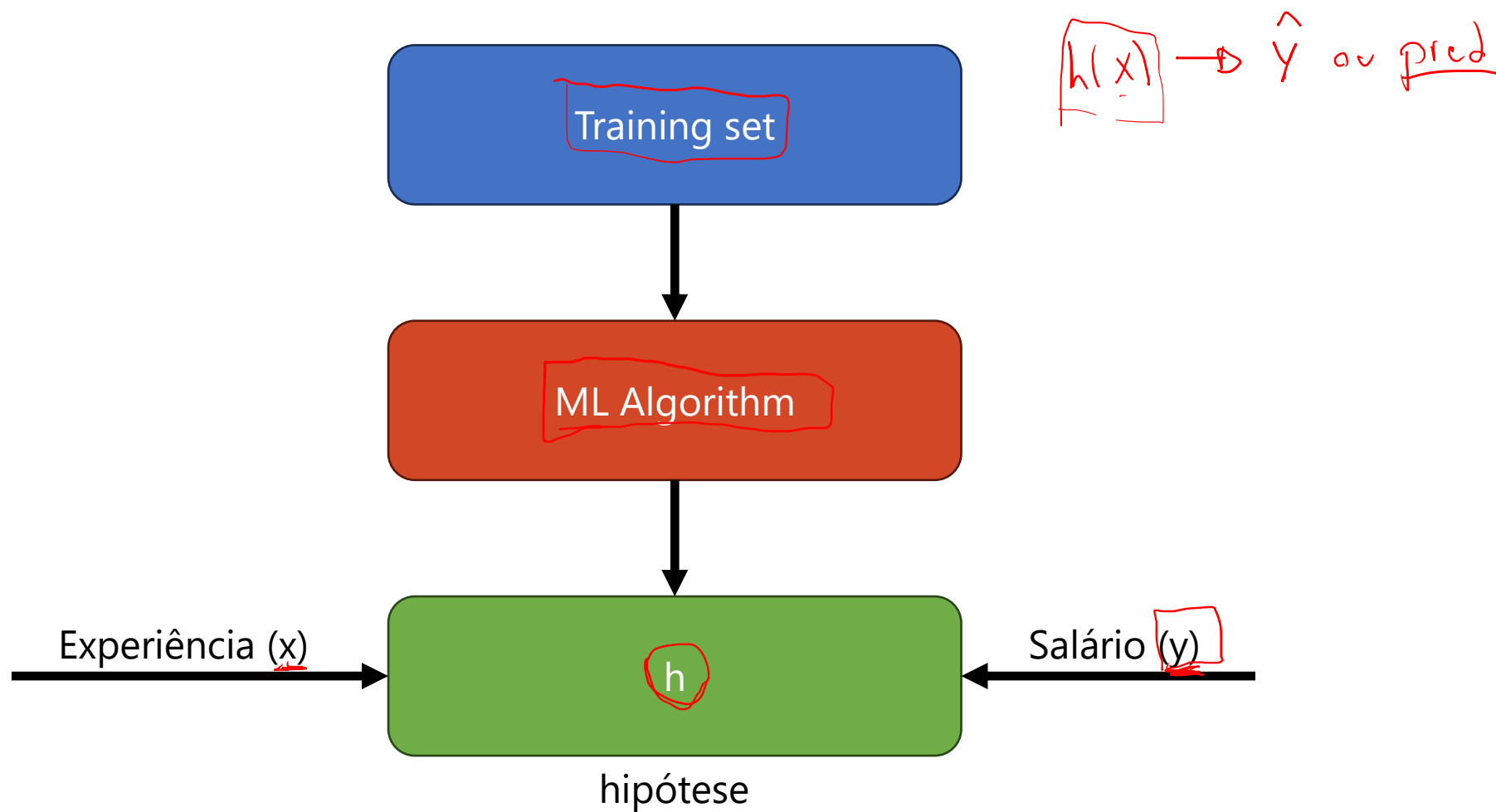
Regressão Linear – Uma variável

Notação:

- **m**: número de exemplos de treinamento
- **x**: número de features {
- **y**: target

Experiência (anos)	Salário Atual (K R\$)
<u>2</u>	<u>36</u>
<u>8</u>	<u>50</u>
<u>13</u>	<u>72</u>
<u>20</u>	<u>98</u>
...	...

Regressão Linear – Treinamento



Regressão Linear – Função de Custo

Notação:

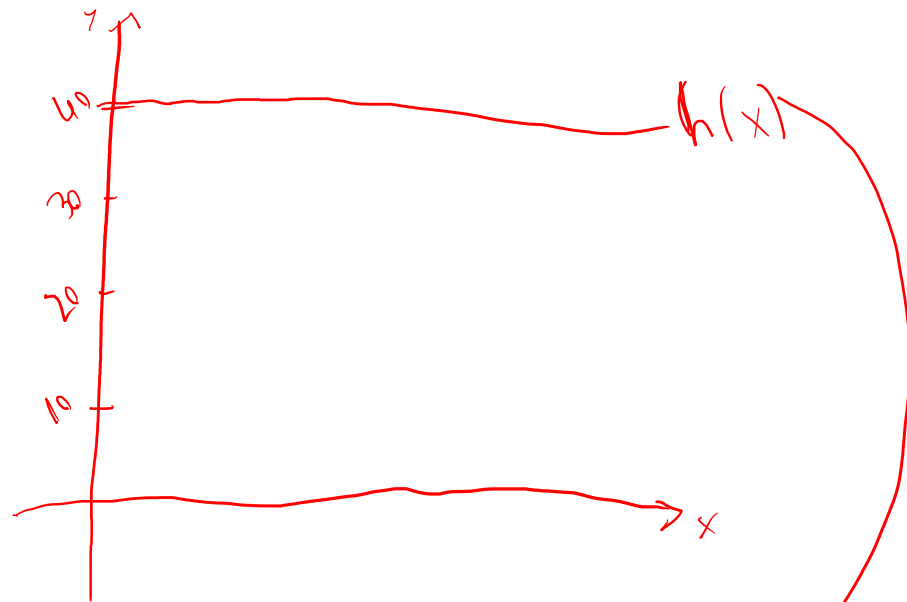
- **m**: número de exemplos de treinamento
- **x**: features
- **y**: target

Experiência (anos)	Salário Atual (K R\$)
2	36
8	50
13	72
20	98
...	...

Hipótese: $h(x) = \theta_0 + \theta_1 \cdot x$

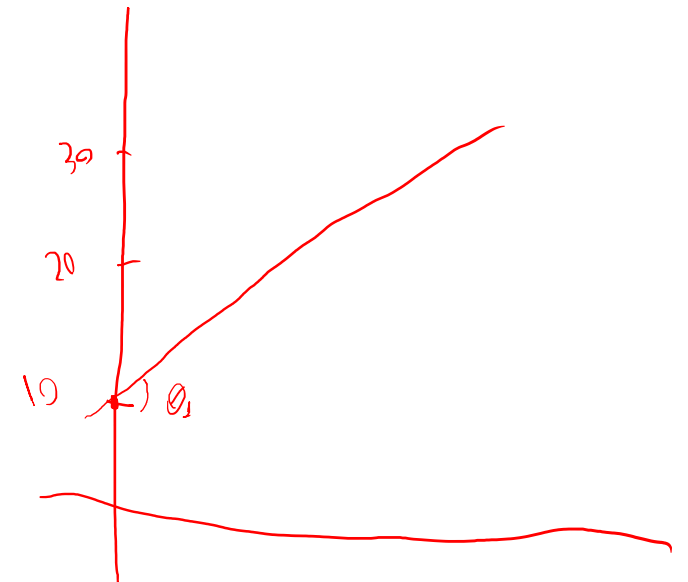
Regressão Linear – Função de Custo

Hipótese: $h(x) = \theta_0 + \theta_1 \cdot x$



$$h(x) = 40_0 + 0_1 \cdot x$$

$$\theta_0 = 40 \mid \theta_1 = 0$$

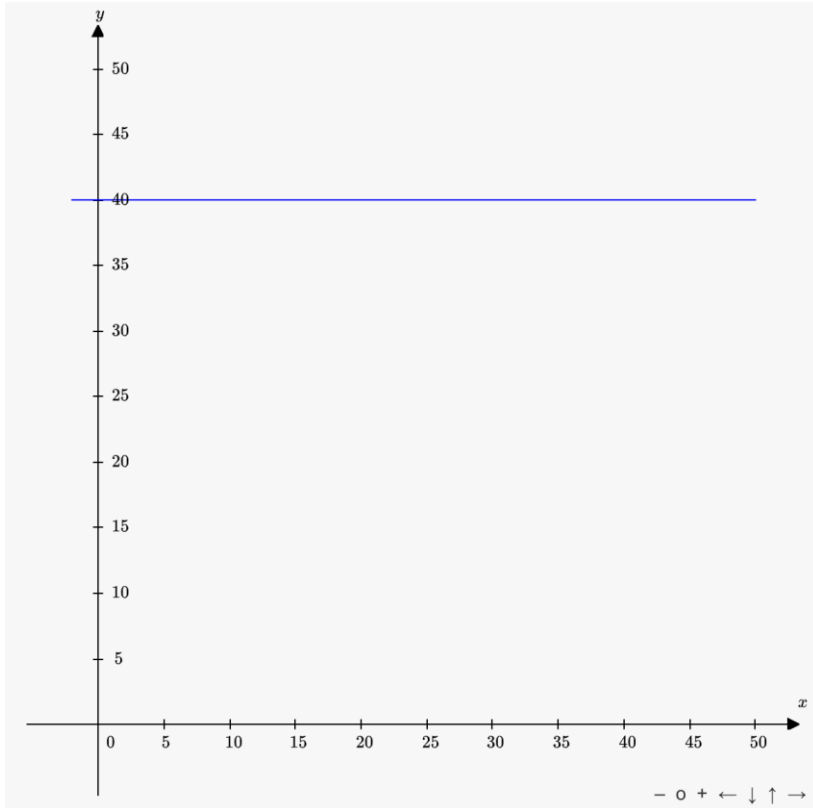


$$h(x) = 10_0 + 0.5_1 \cdot x$$

$$\theta_0 = 10 \mid \theta_1 = 0.5$$

Regressão Linear – Função de Custo

Hipótese: $h(x) = \theta_0 + \theta_1 \cdot x$



$$h(x) = 40_0 + 0_1 \cdot x$$

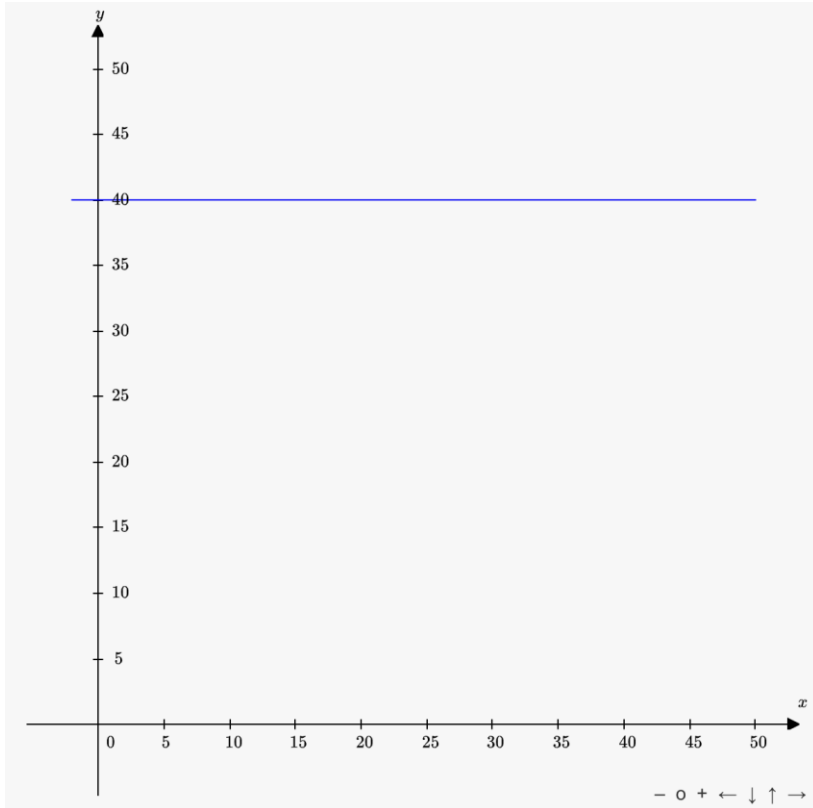
$$\theta_0 = 40 \mid \theta_1 = 0$$

$$h(x) = 10_0 + 0.5_1 \cdot x$$

$$\theta_0 = 10 \mid \theta_1 = 0.5$$

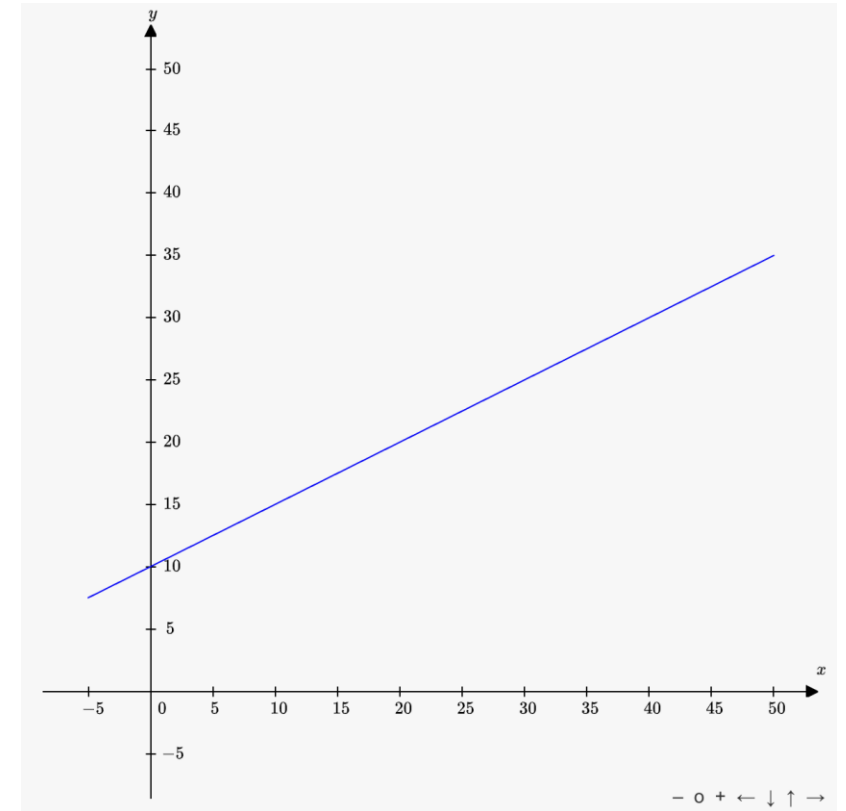
Regressão Linear – Função de Custo

Hipótese: $h(x) = \theta_0 + \theta_1 \cdot x$



$$h(x) = 40_0 + 0_1 \cdot x$$

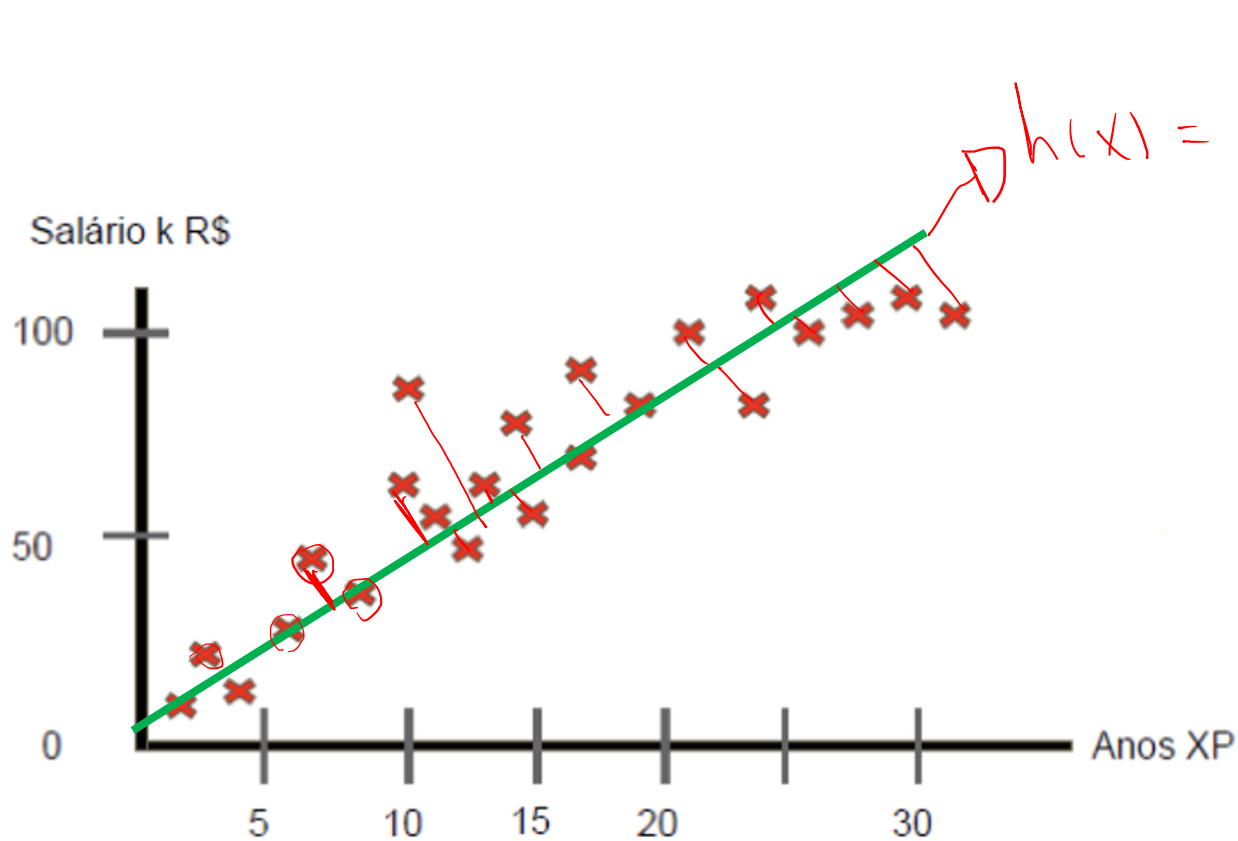
$$\theta_0 = 40 \mid \theta_1 = 0$$



$$h(x) = 10_0 + 0.5_1 \cdot x$$

$$\theta_0 = 10 \mid \theta_1 = 0.5$$

Regressão Linear - Função de Custo

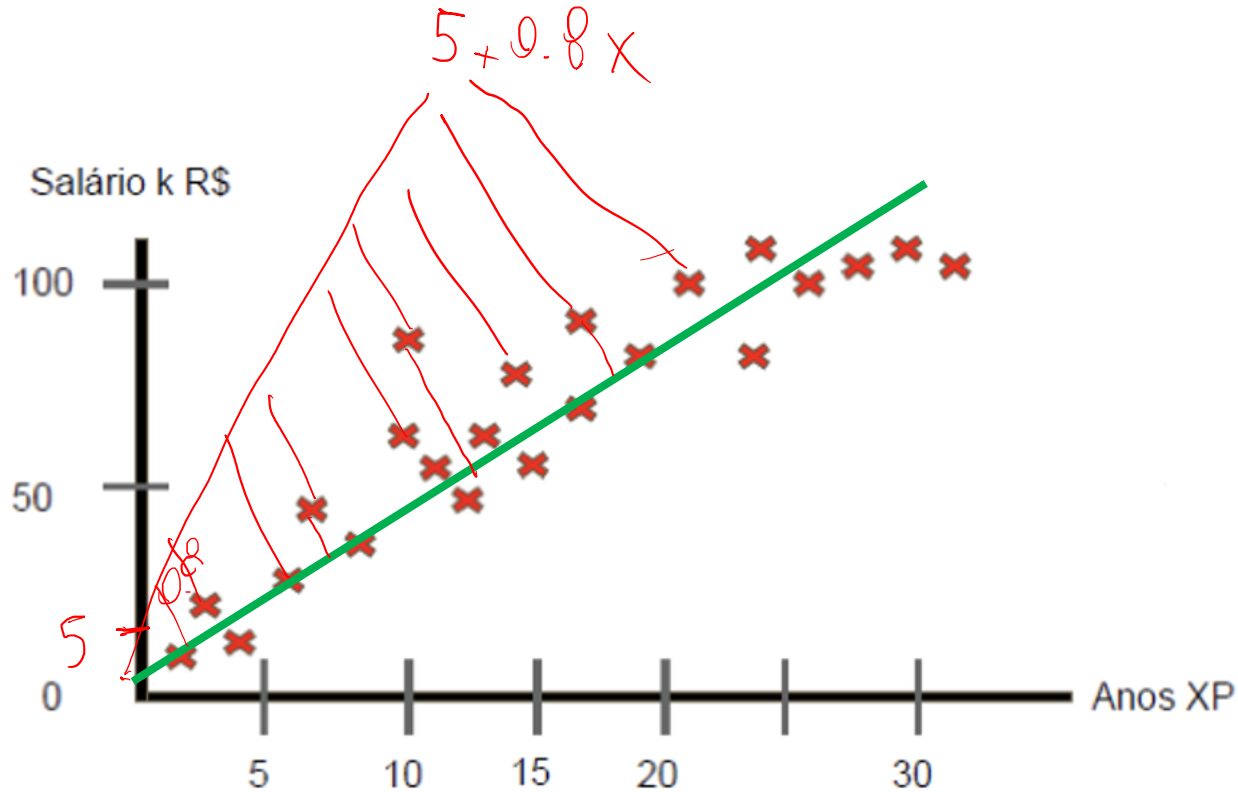


$$h(x) = \theta_0 + \theta_1 \cdot x = \boxed{\hat{y}} \rightarrow \text{Pred}$$

$\neq$
 y

Escolher θ_0, θ_1 para que $h(x)$ seja perto de y com base no set de treinamento.

Regressão Linear - Função de Custo



Escolher θ_0, θ_1 para que $h(x)$ seja perto de y com base no set de treinamento.

Hipótese: $h(x) = \theta_0 + \theta_1 \cdot x$

Minimize: θ_0, θ_1

Mean Square Error Function

$$(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + \dots + (y_n - \hat{y}_n)^2 = \sum_{i=1}^n (y^i - \hat{y}^i)^2$$

↖ MSE

Regressão Linear – Gradiente Descendente

Objetivo: $h(x) = \theta_0 + \theta_1 \cdot x$

- Minimizar θ_0, θ_1

$\theta_0, \theta_1 \rightarrow w_0, w_1$ $\rightarrow W \rightarrow \text{Weight}$

1 $\rightarrow \theta_0, \theta_1$

2 $\rightarrow h(x) = \theta_0 + \theta_1 x = \hat{y}$

3 $\rightarrow \text{MSE} \rightarrow \text{Erro}$

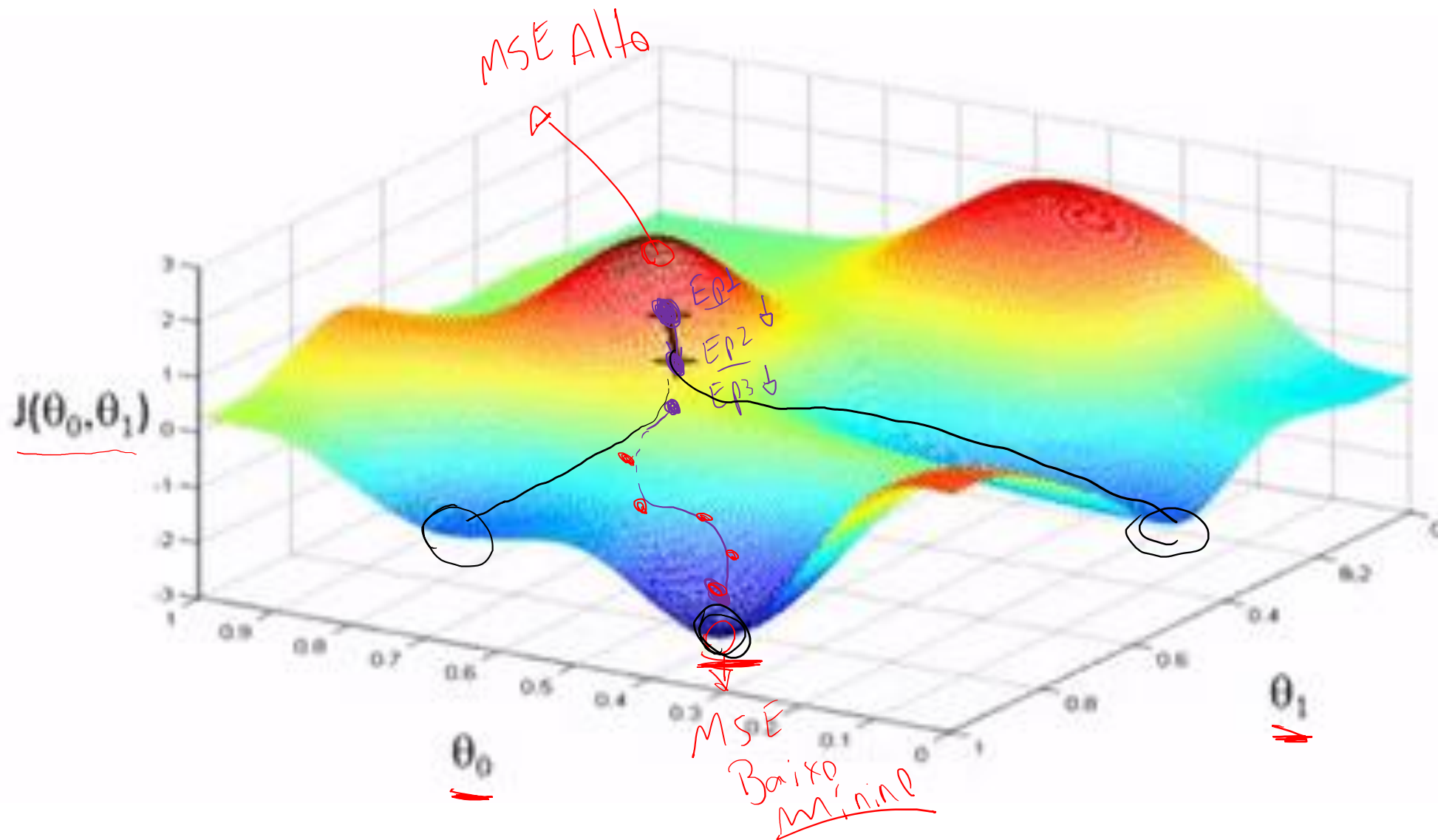
Como Fazer:

- Começar com um θ_0, θ_1 qualquer
- Mudar os valores de θ_0, θ_1 para reduzir o erro, até que se chegue a um mínimo

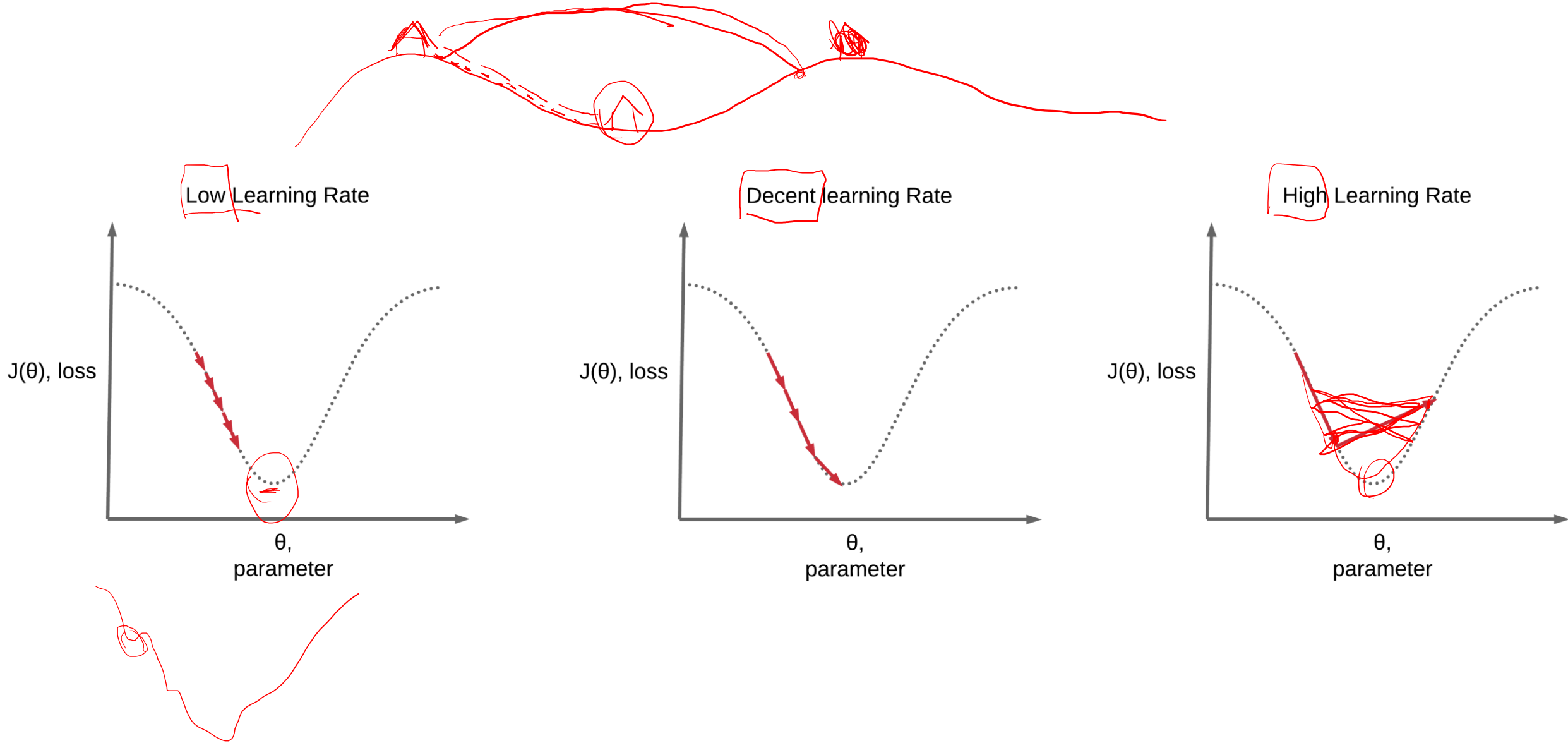
$\rightarrow \text{MSE}$
Mean Square Error Function

$$\sum_{i=1}^n (y^i, \hat{y}^i)^2$$

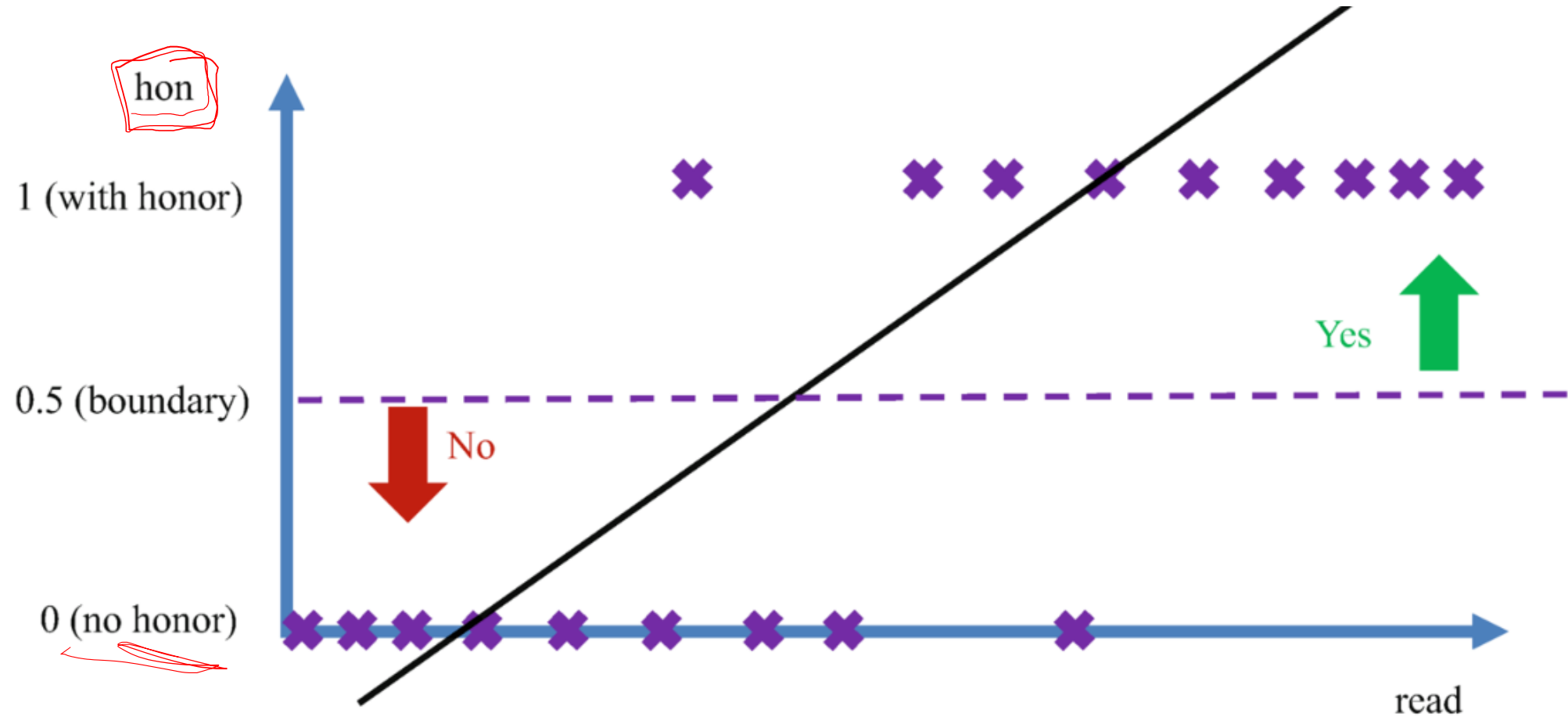
Regressão Linear – Gradiente Descendente



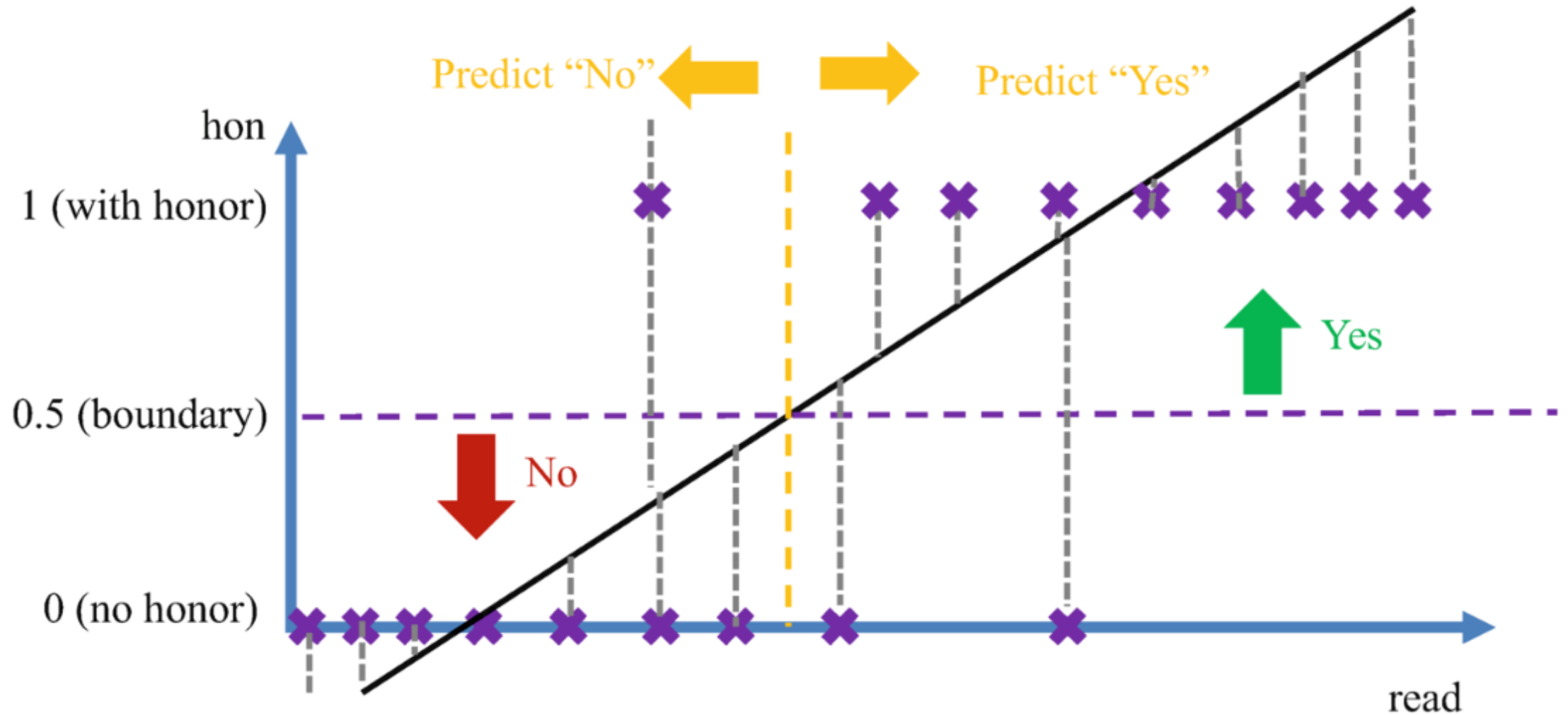
Regressão Linear – Learning Rate



Regressão Logística

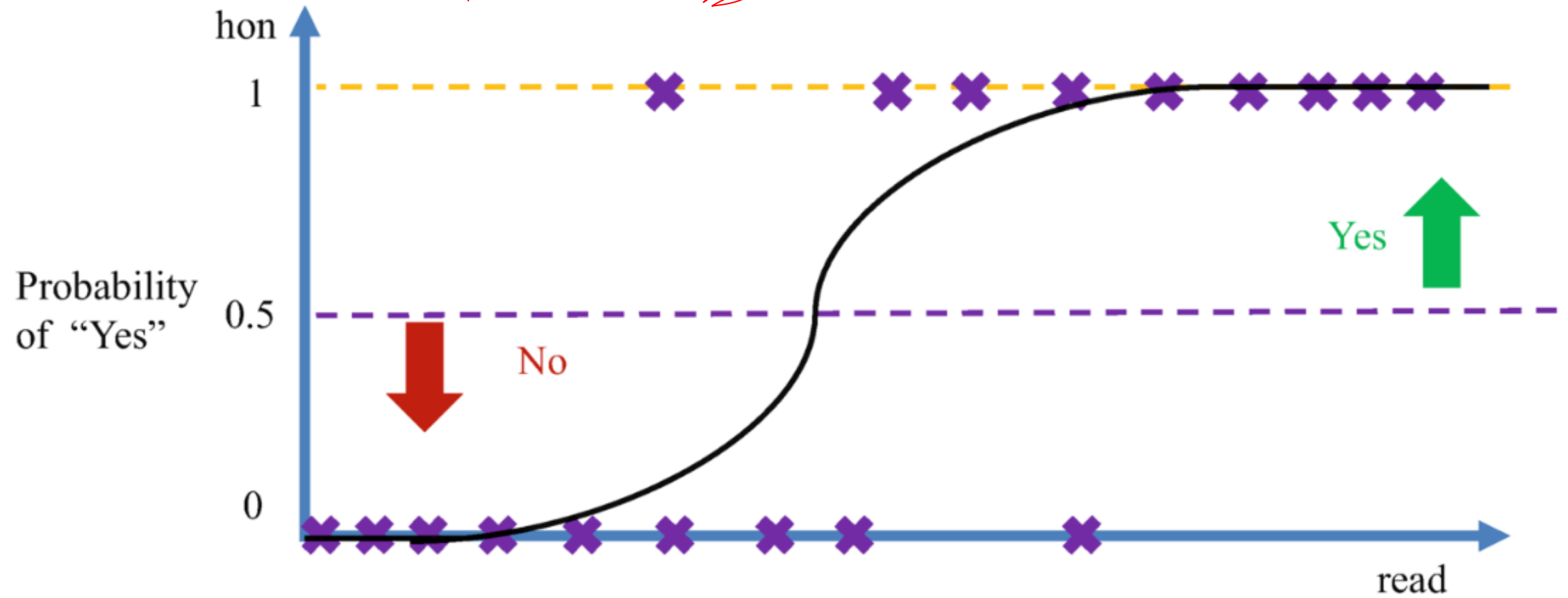


Regressão Logística

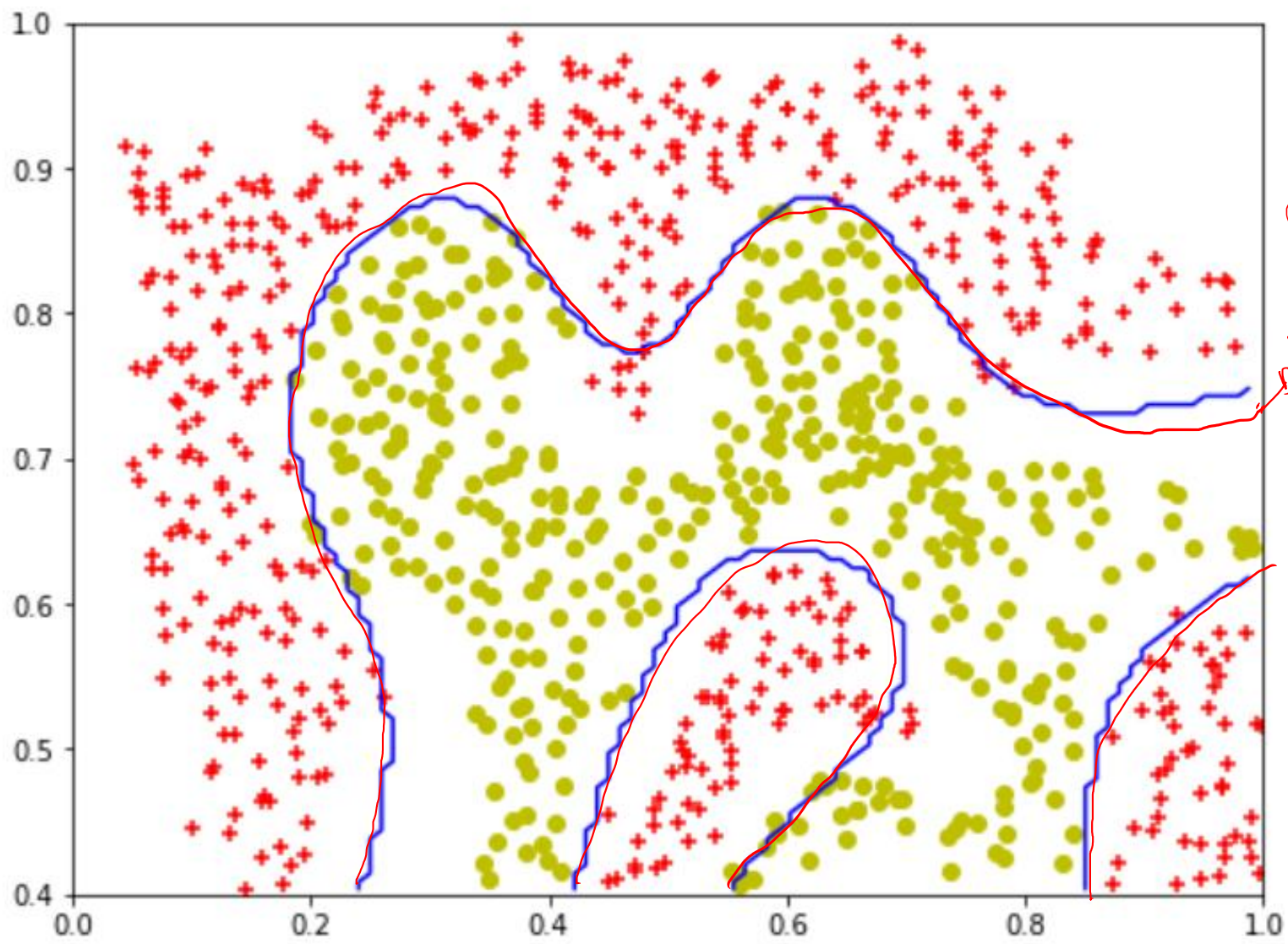
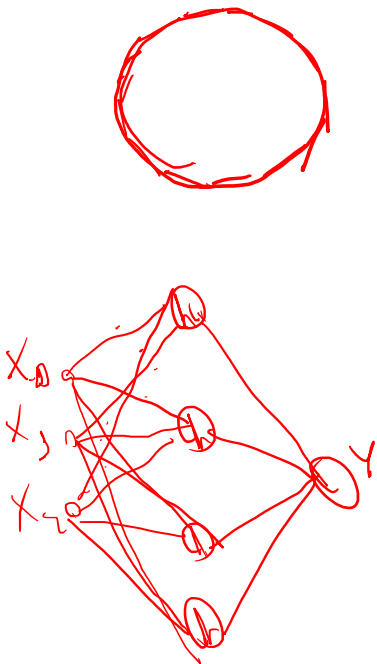
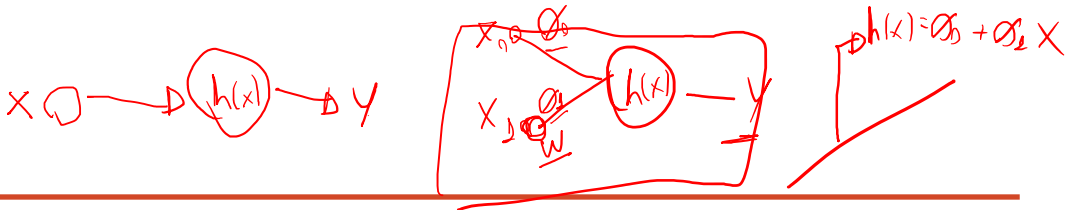


Regressão Logística

Não usa MSE ~~Mean Square Error~~



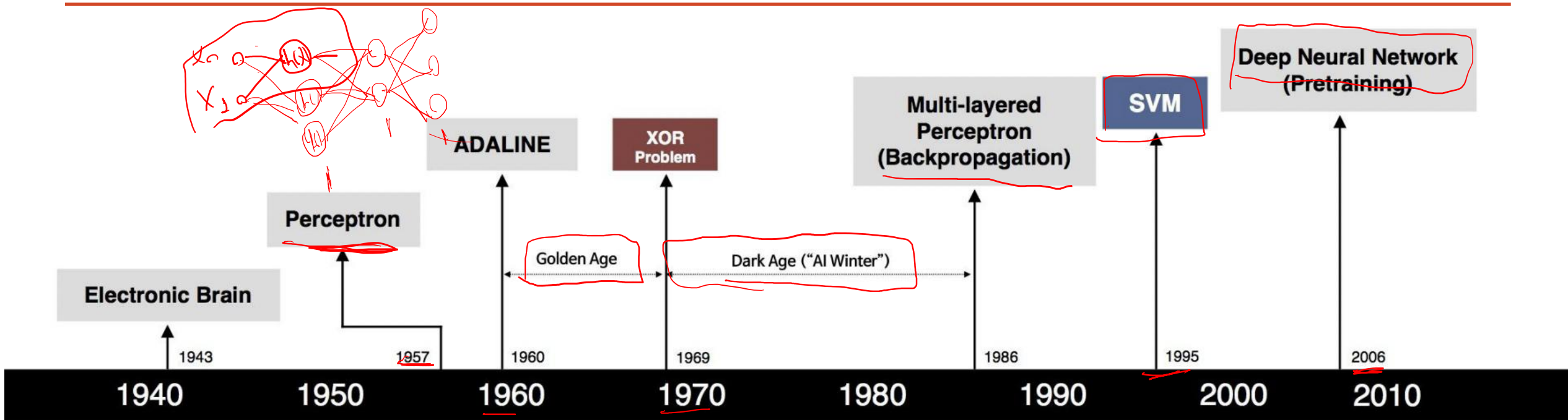
Redes Neurais - Hipótese não-linear



$\phi_0, \phi_1, \phi_2, \dots, \phi_n$

$h(x)$

Redes Neurais - História



S. McCulloch - W. Pitts



F. Rosenblatt



B. Widrow - M. Hoff



M. Minsky - S. Papert



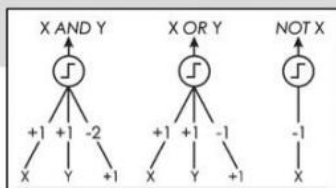
D. Rumelhart - G. Hinton - R. Williams



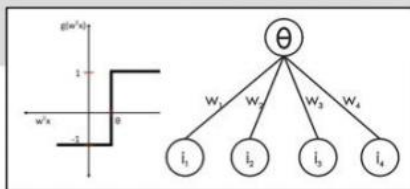
V. Vapnik - C. Cortes



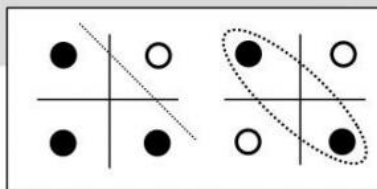
G. Hinton - S. Ruslan



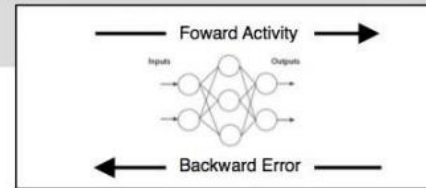
- Adjustable Weights
- Weights are not Learned



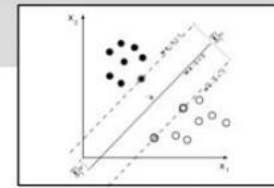
- Learnable Weights and Threshold



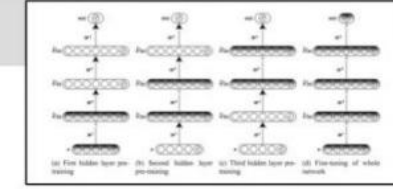
- XOR Problem



- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting

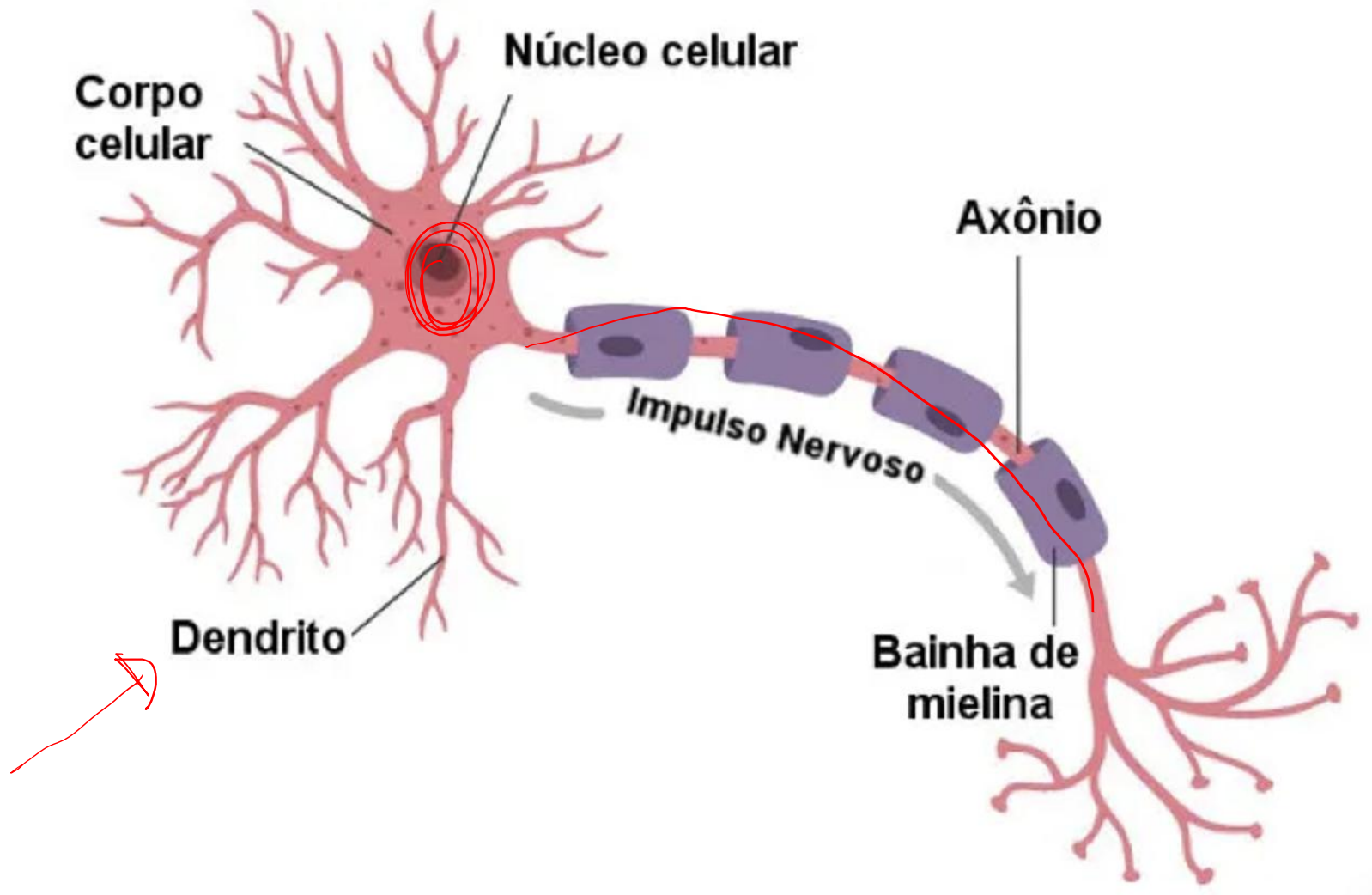


- Limitations of learning prior knowledge
- Kernel function: Human Intervention



- Hierarchical feature Learning

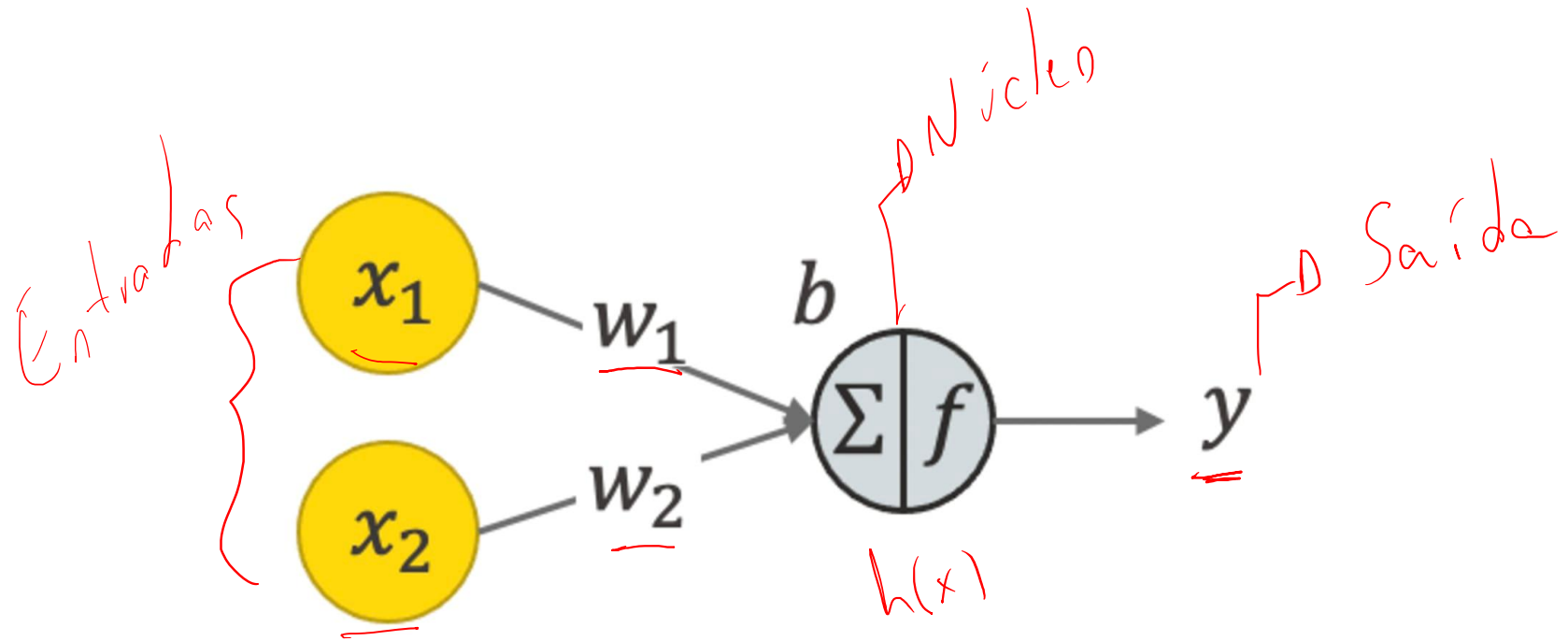
Redes Neurais - Neurônio



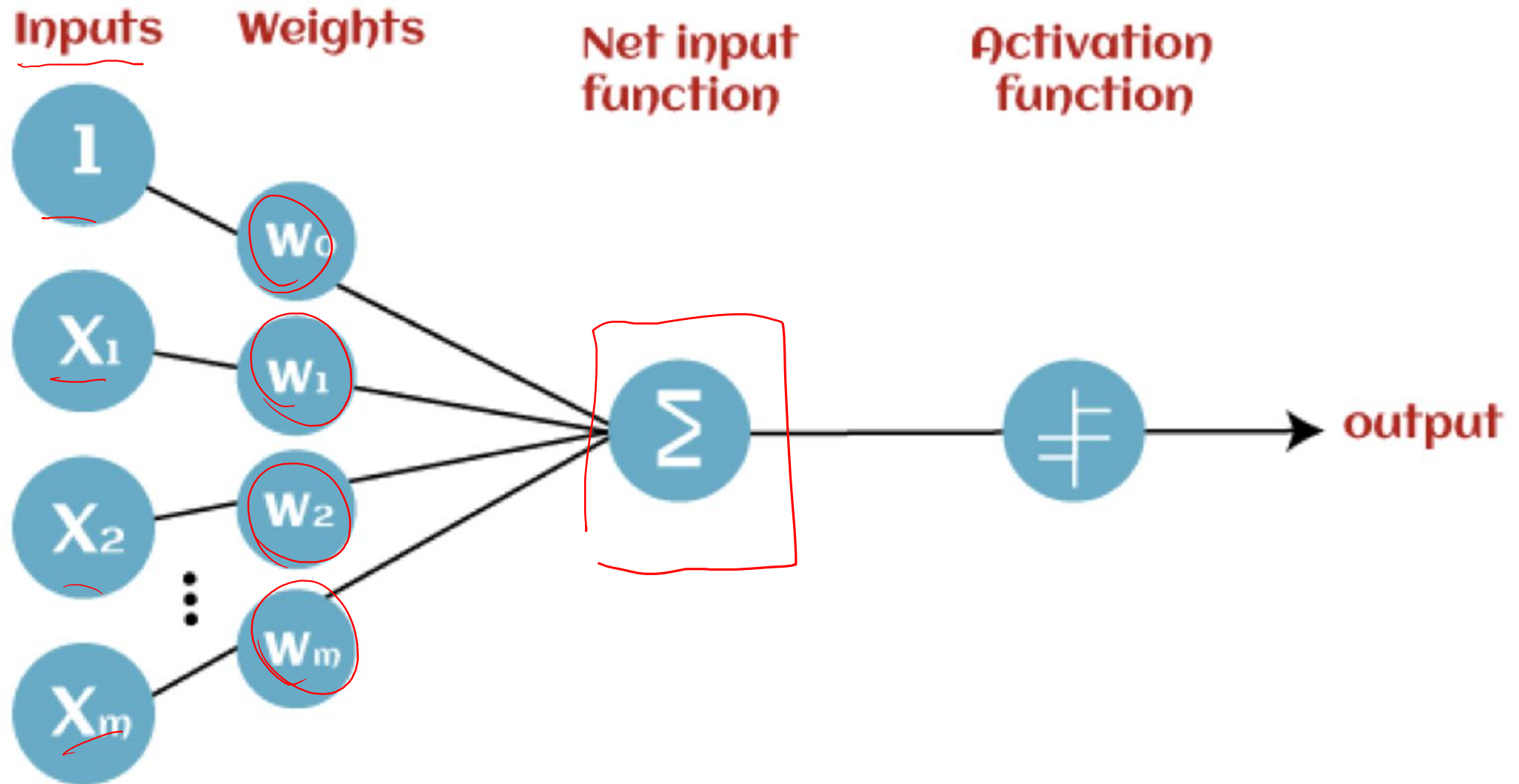
Redes Neurais - Perceptron

X

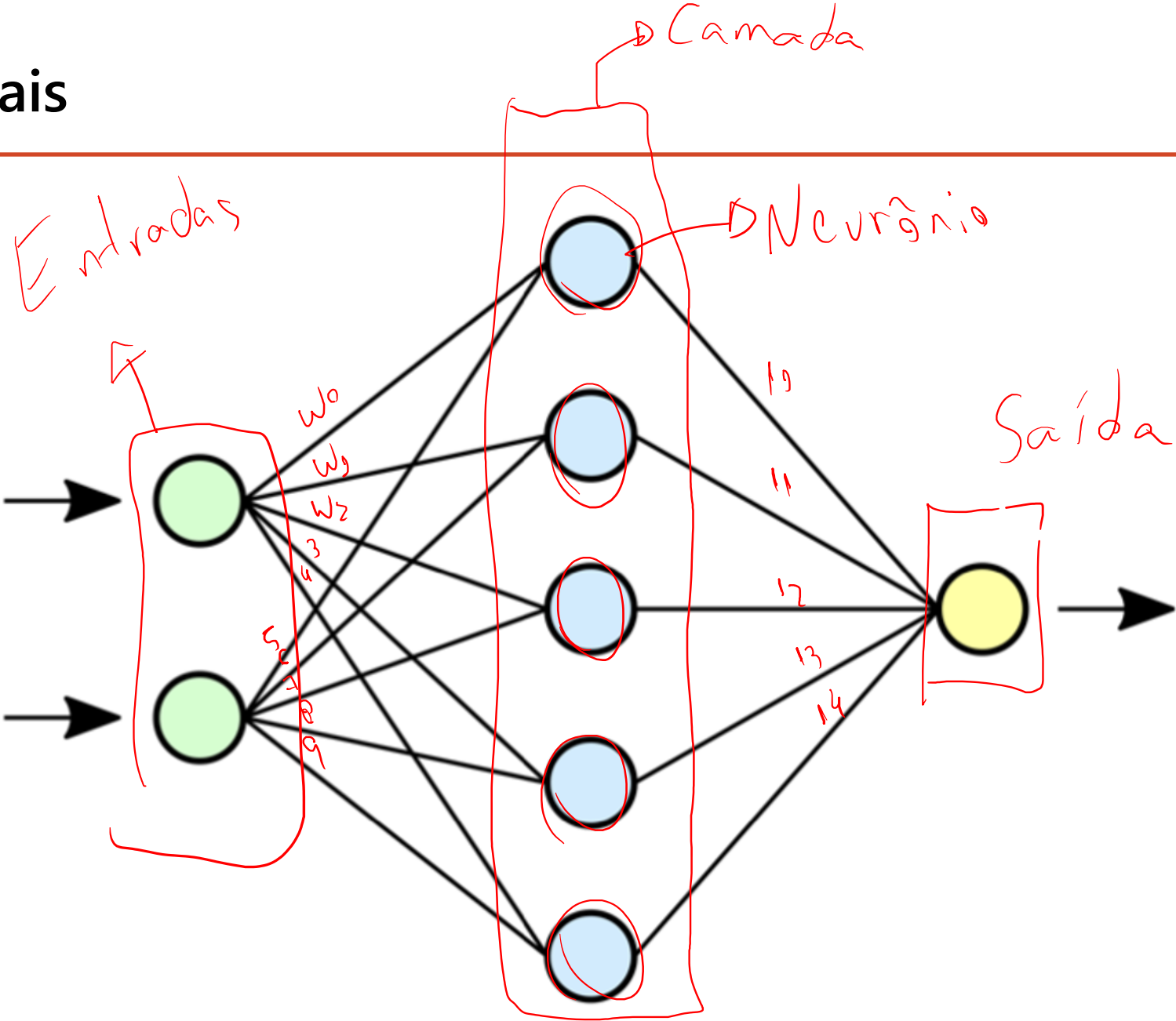
x_{00}	x_{01}
x_{02}	x_{03}
x_{02}	x_{02}
$x_{\dots}$	$x_{\dots}$



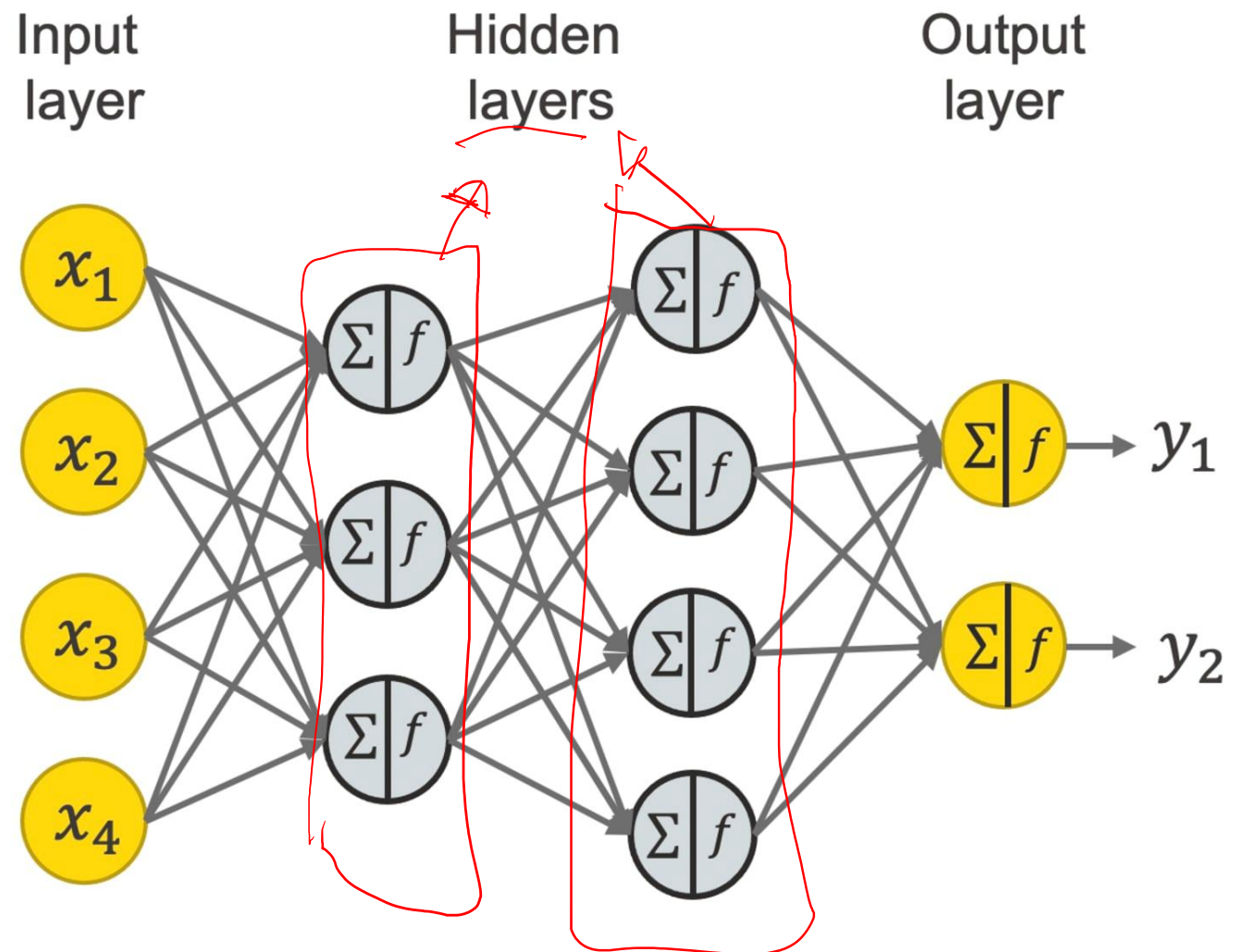
Redes Neurais – Perceptron



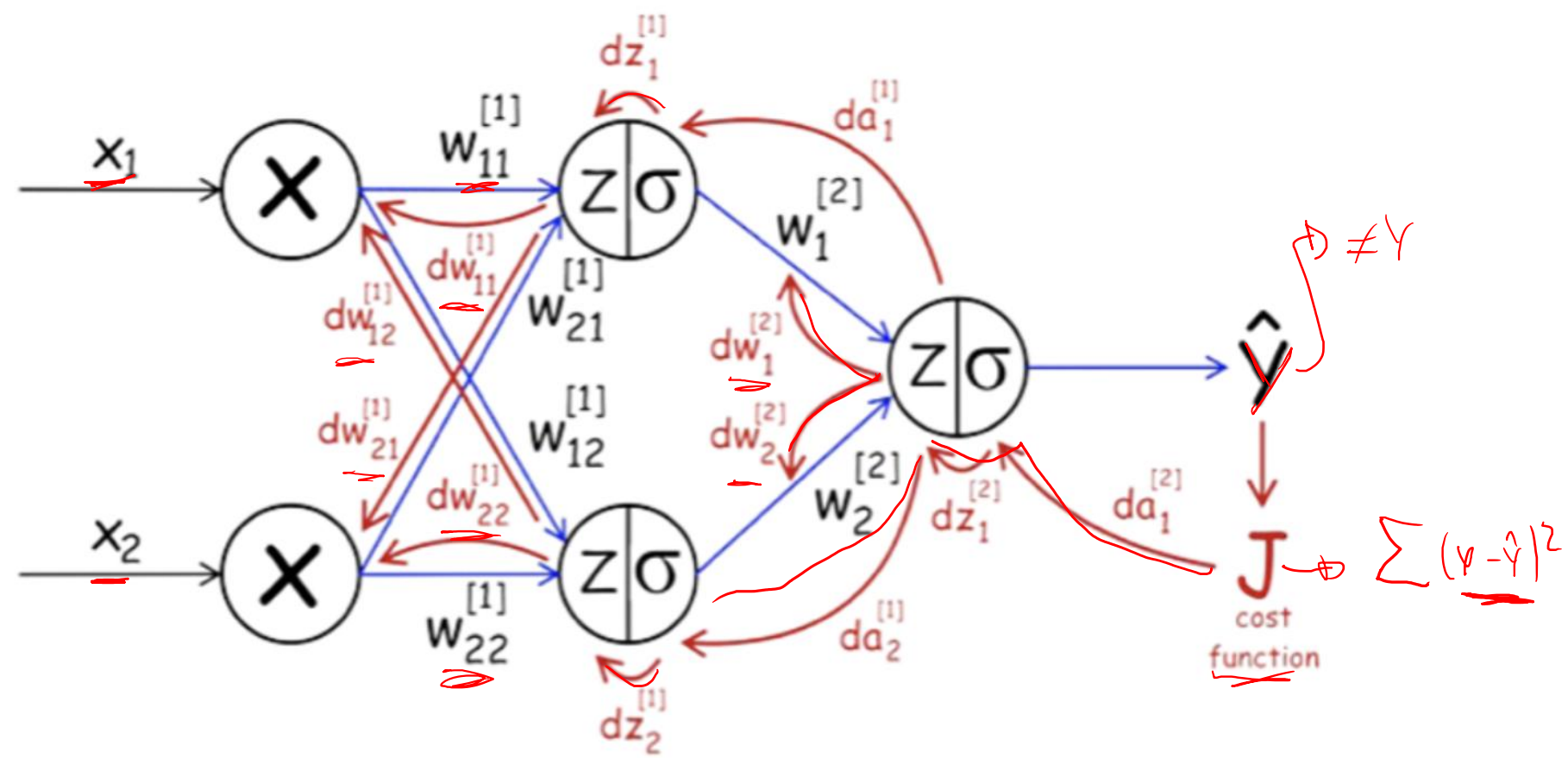
Redes Neurais



Redes Neurais



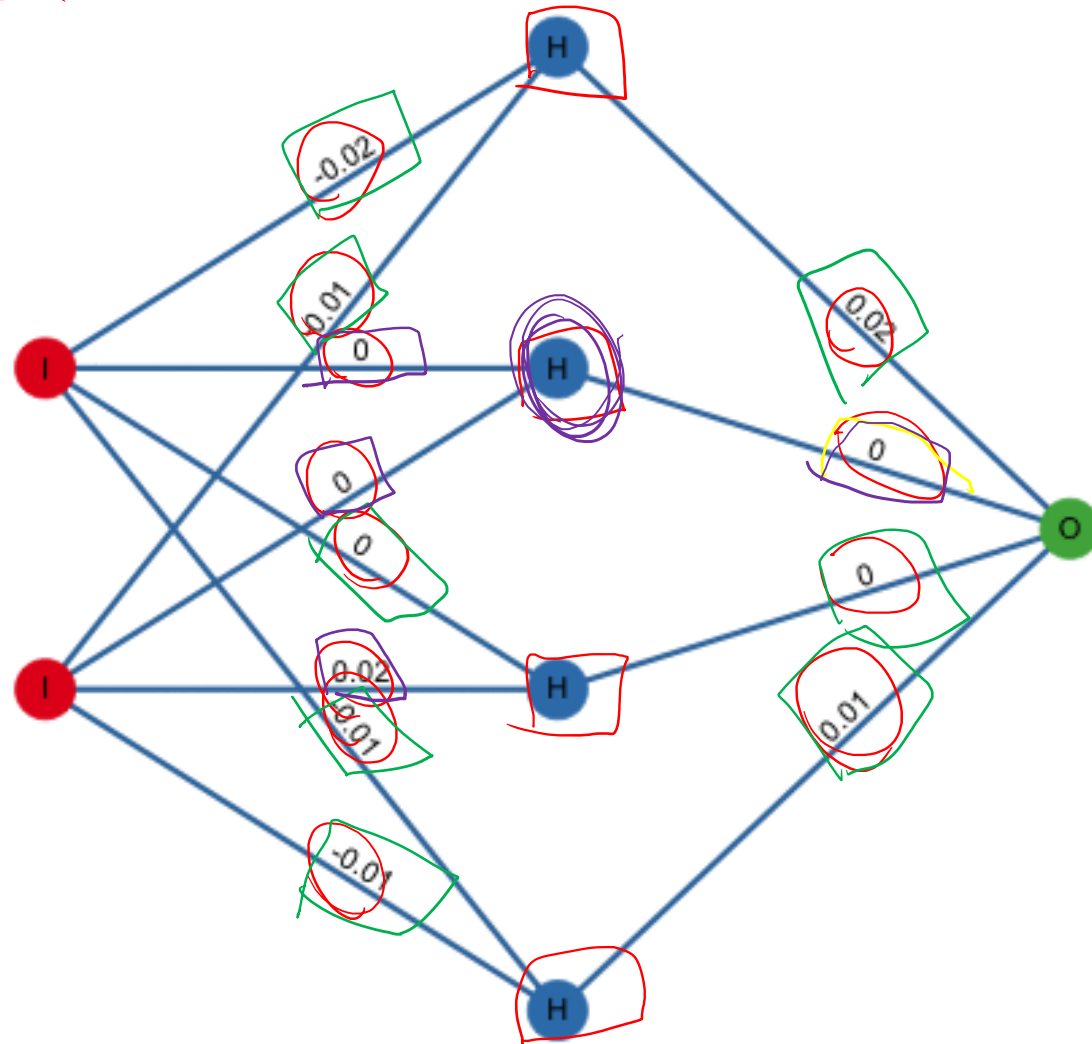
Redes Neurais - Backpropagation



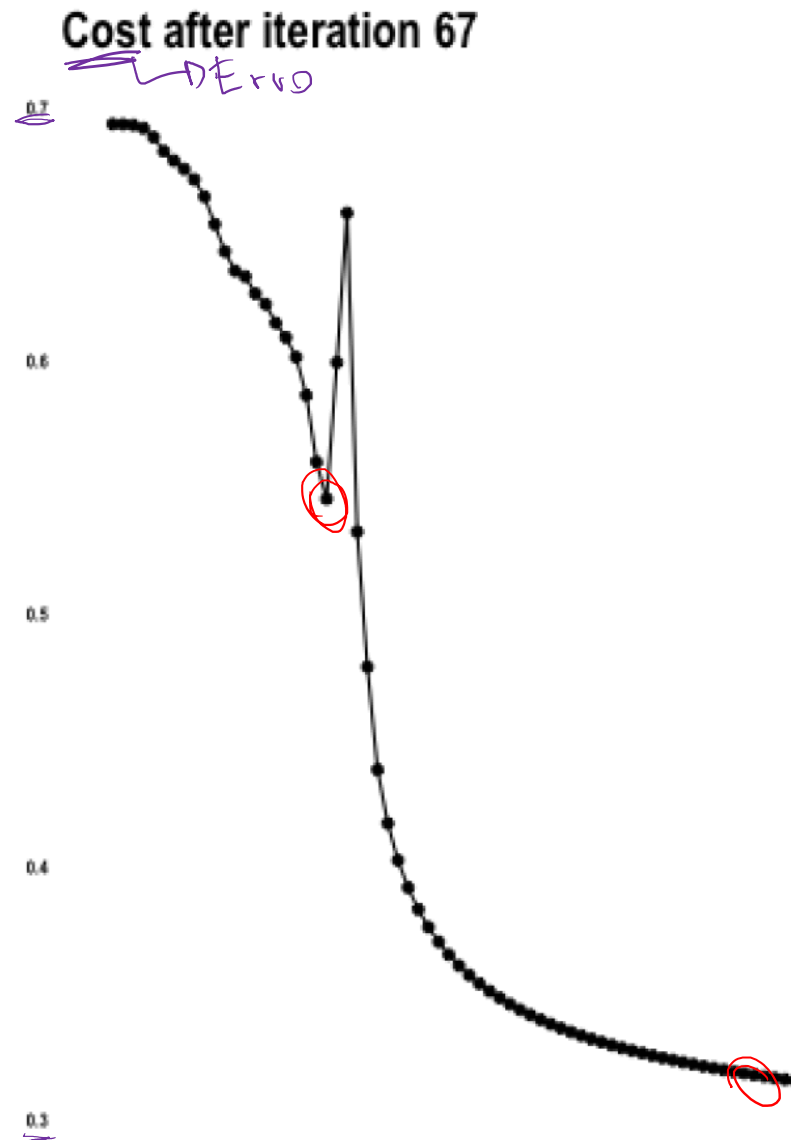
Backpropagation

Redes Neurais - Training

Weights after iteration 0

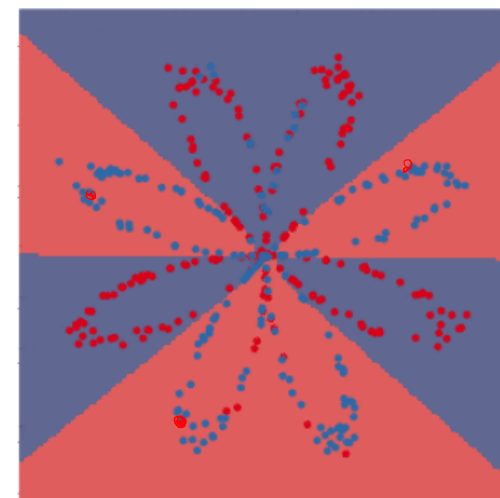
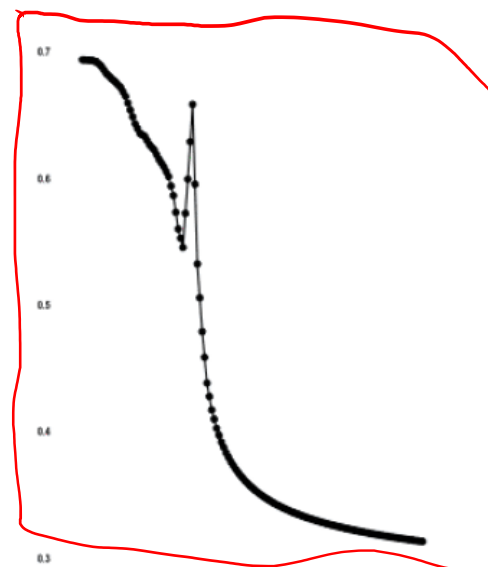
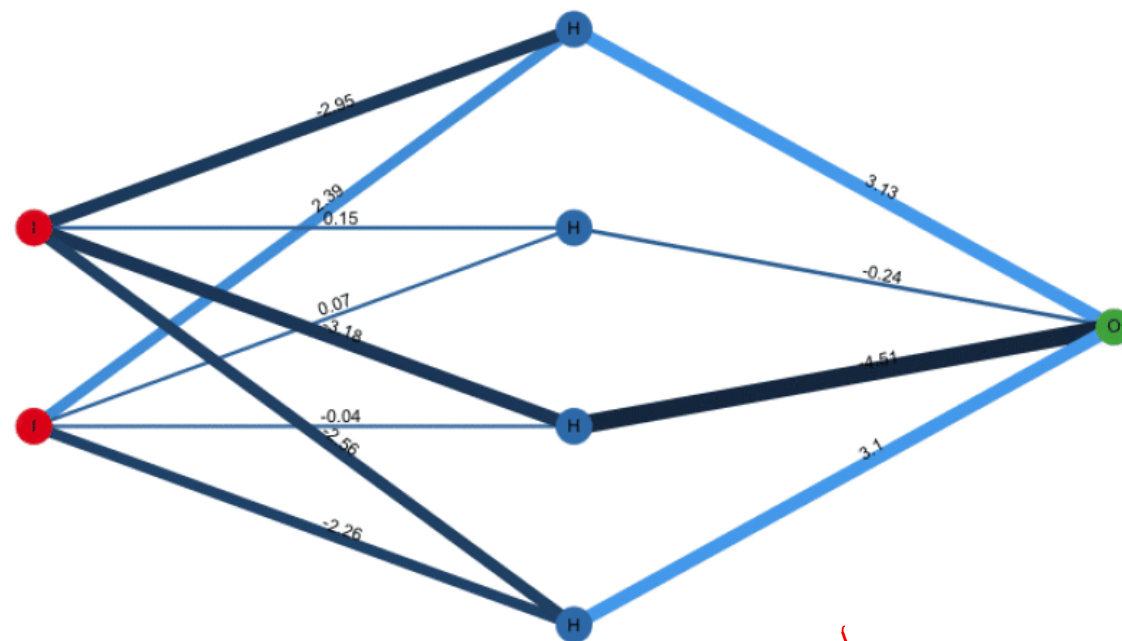


Redes Neurais - Training



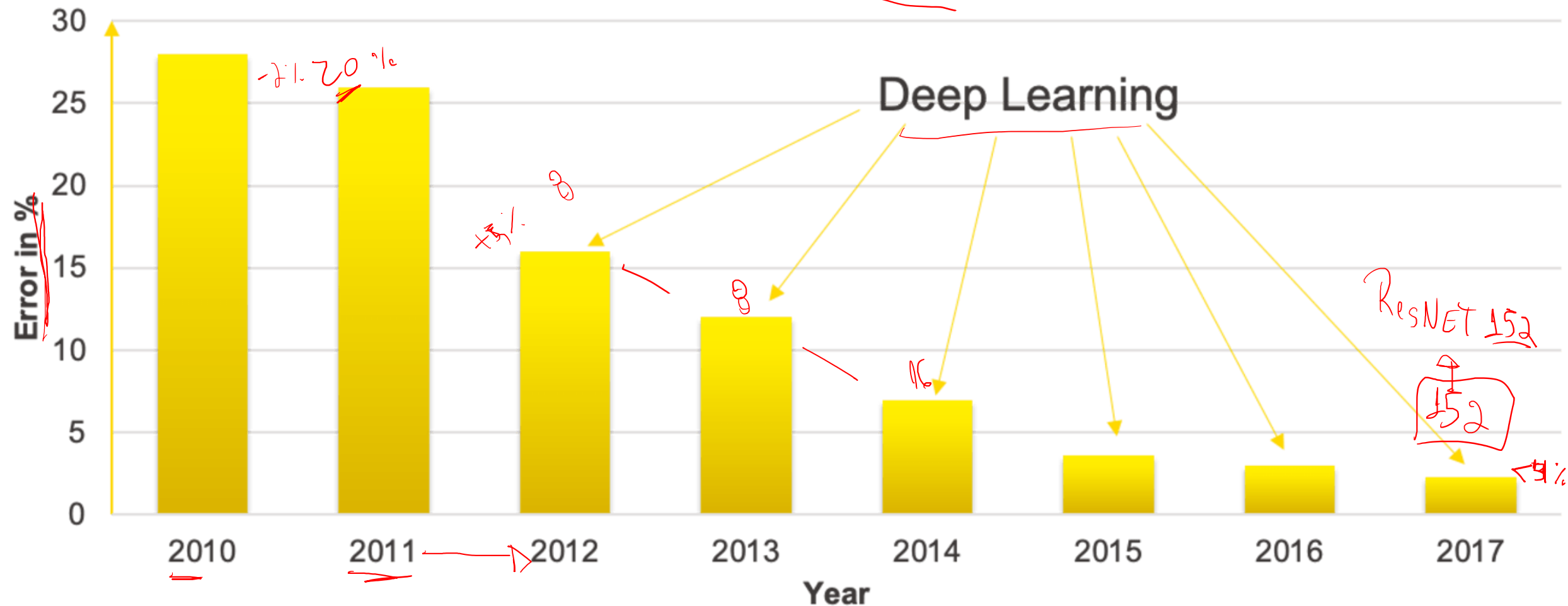
Redes Neurais - Training

Training a neural net at iteration 71

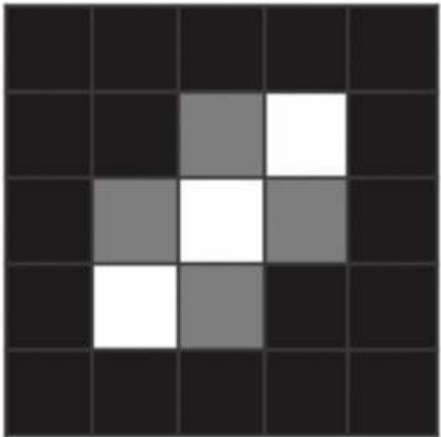


Deep Learning

Winners of the ImageNet Challenge



Deep Learning



=

-1	-1	-1	-1	-1
-1	-1	0.2	1	-1
-1	0.2	1	0.2	-1
-1	1	0.2	-1	-1
-1	-1	-1	-1	-1



=

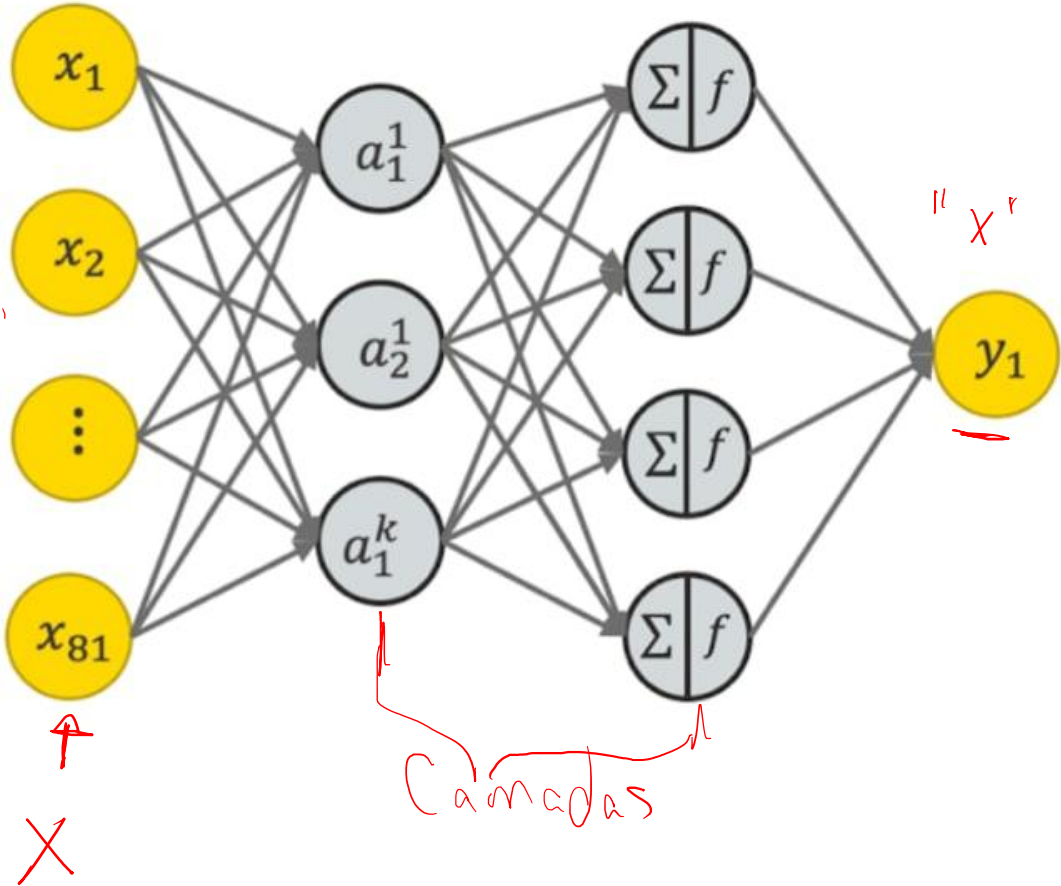
0.8	0.6	...	0.6	0.9		
0.1	0.5	...	0.5	0.2		
⋮	0.2	0.1	0.1	...	0.1	0.3
0.8	⋮	0.3	0.2	...	0.1	0.1
0.6	0.1	⋮	0.5	⋮	⋮	⋮
	0.2	0.8	0.7	...	0.8	0.8
		0.8	0.6	...	0.6	0.9

Deep Learning

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



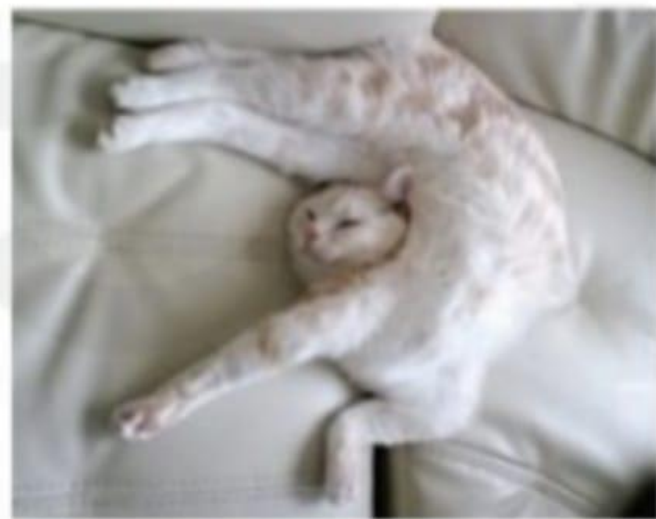
$$\begin{pmatrix} -1 \\ -1 \\ \vdots \\ 1 \\ -1 \\ \vdots \\ -1 \\ -1 \end{pmatrix}$$



Viewpoint variations



Deformations



Illumination conditions

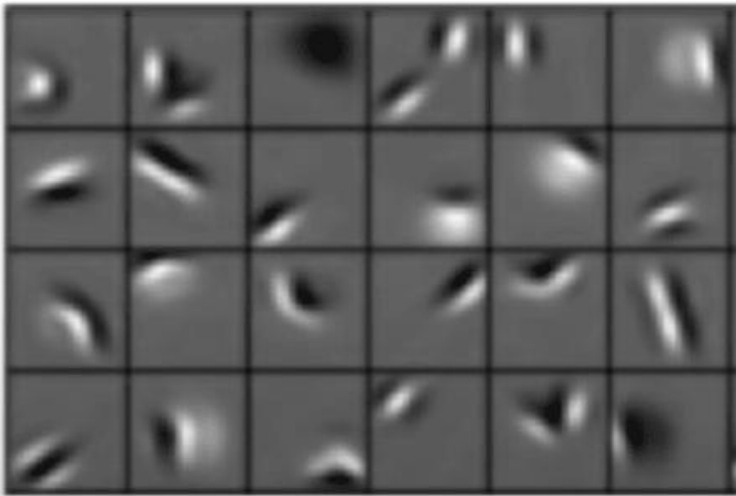


Intra-class variations



Deep Learning

Low level features



Edges, dark spots

Mid level features



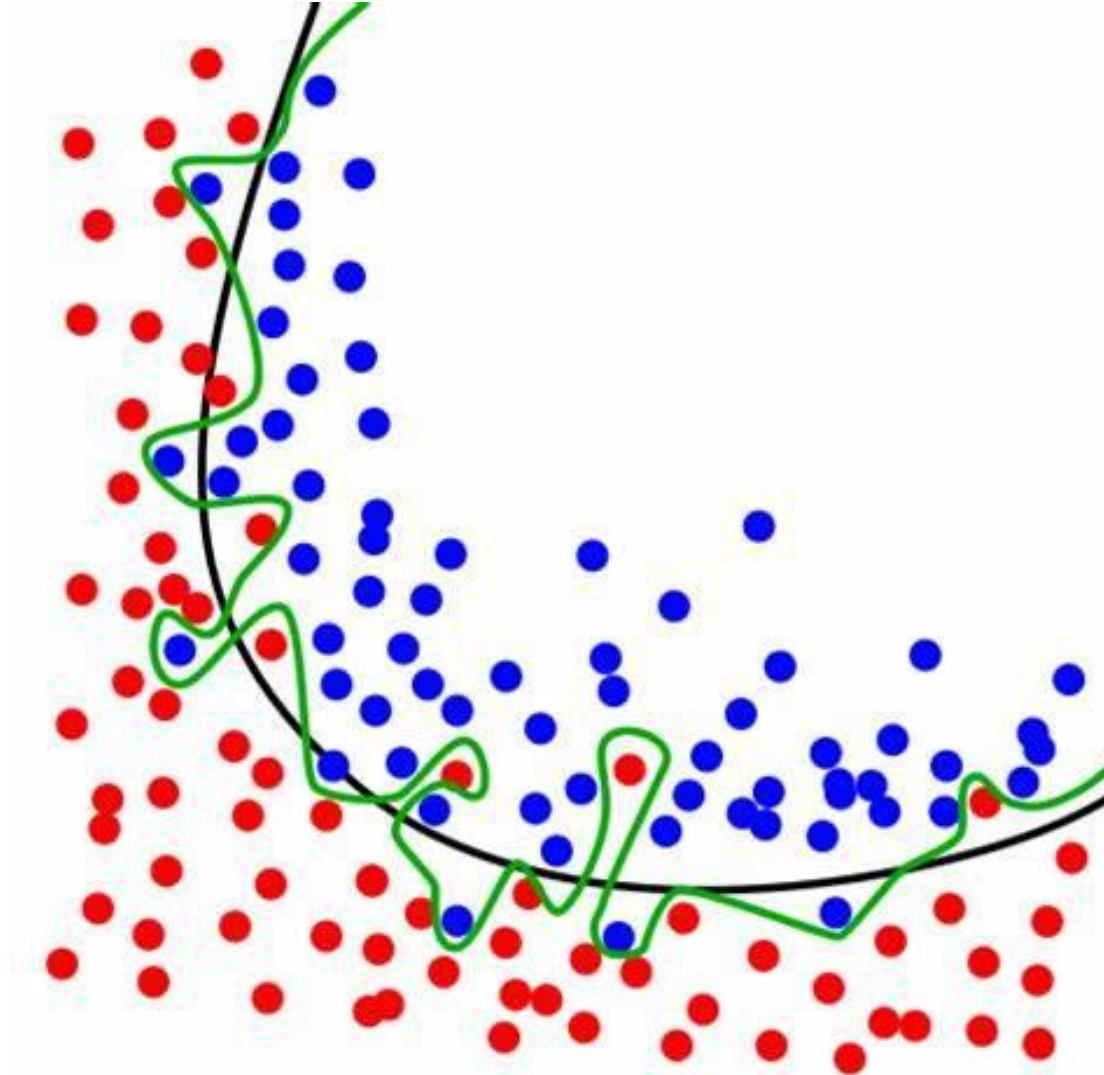
Eyes, ears, nose

High level features

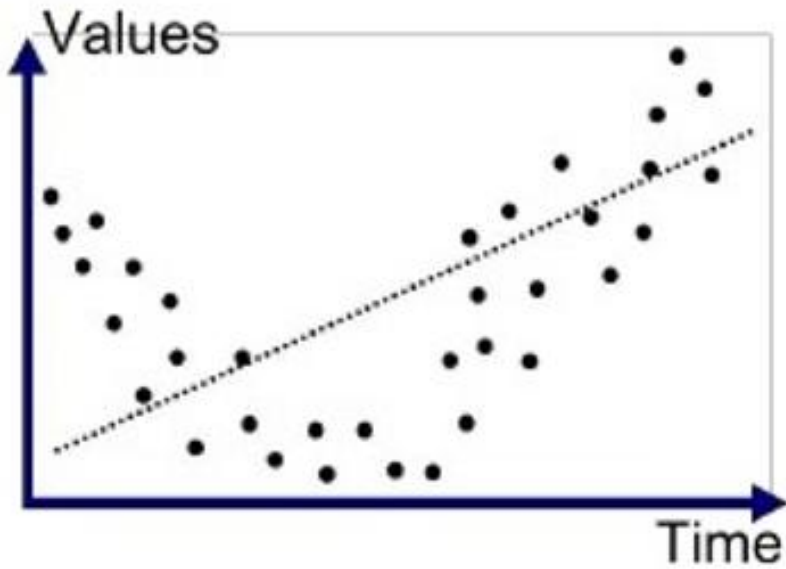


Facial structure

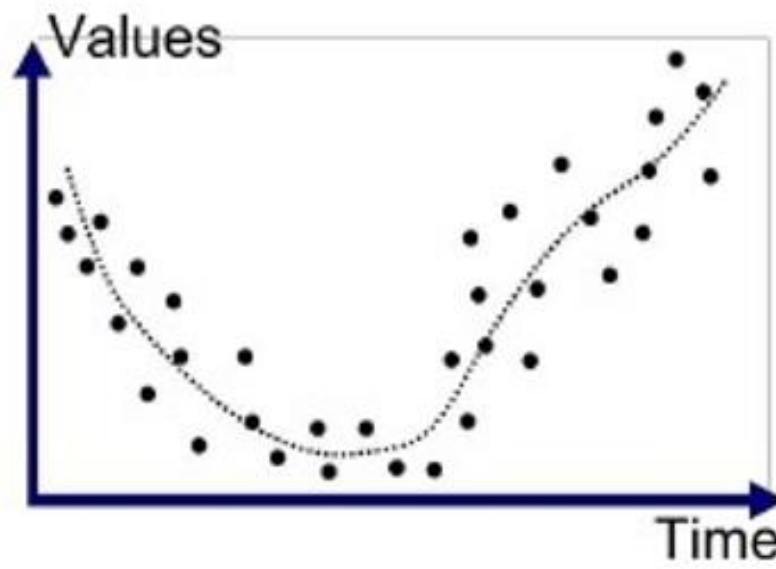
Overfitting e Underfitting



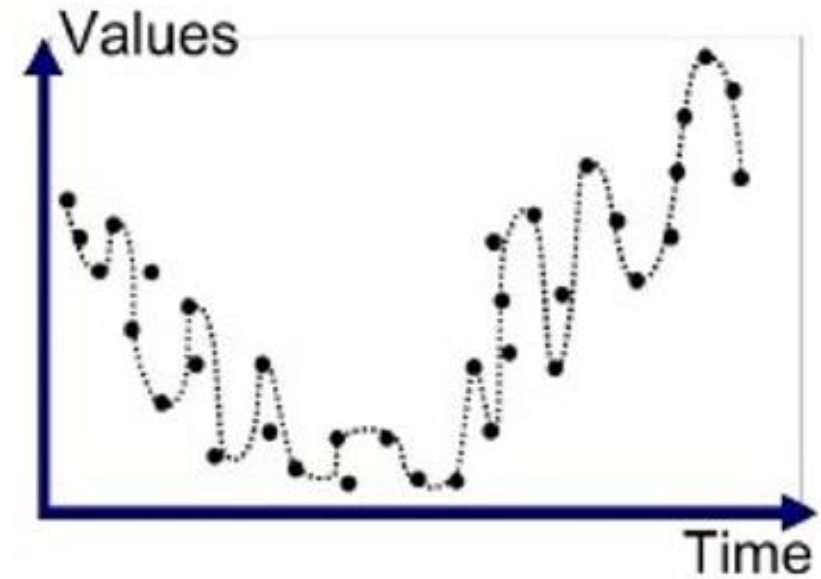
Overfitting e Underfitting



Underfitted

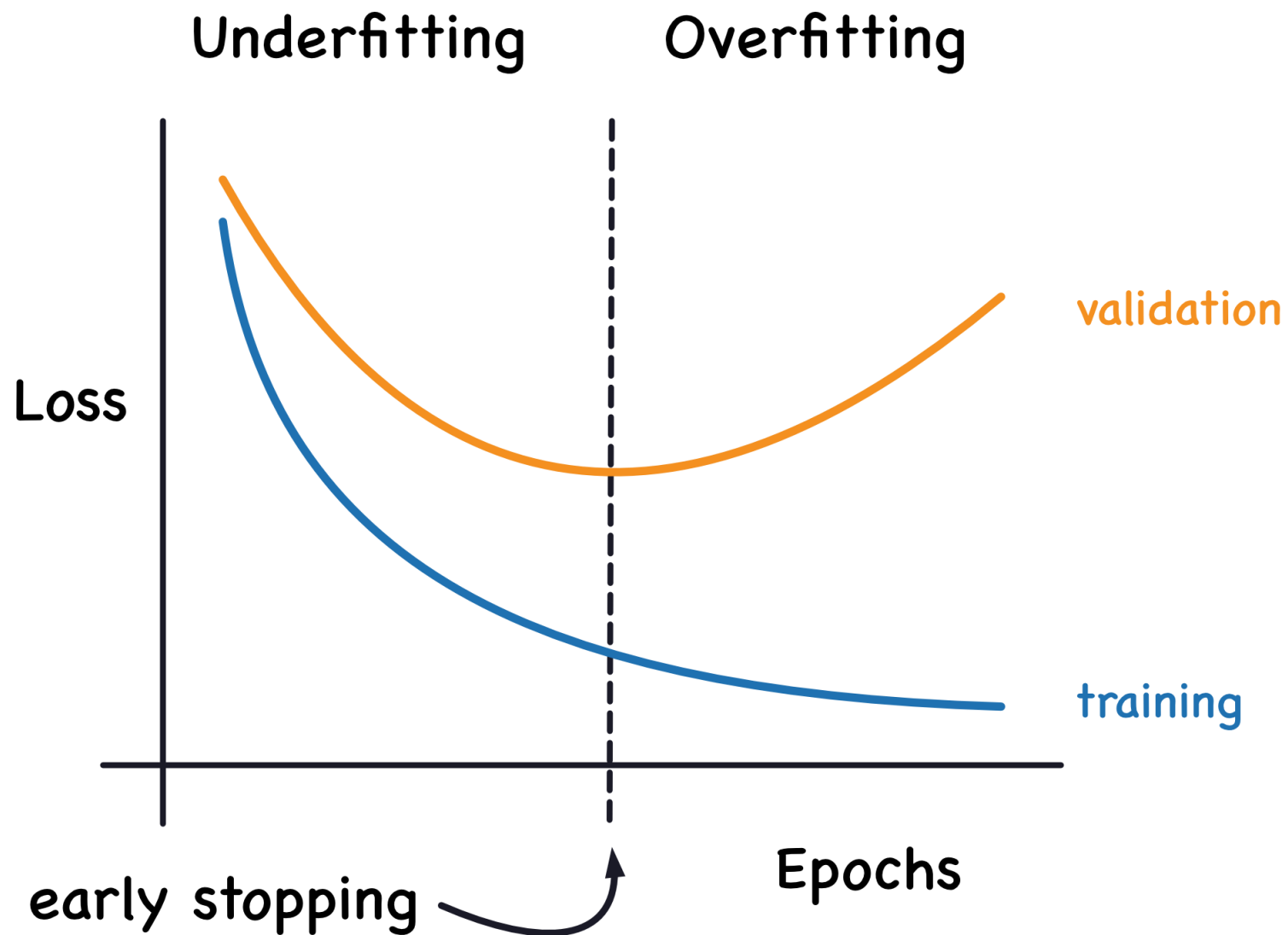


Good Fit/Robust

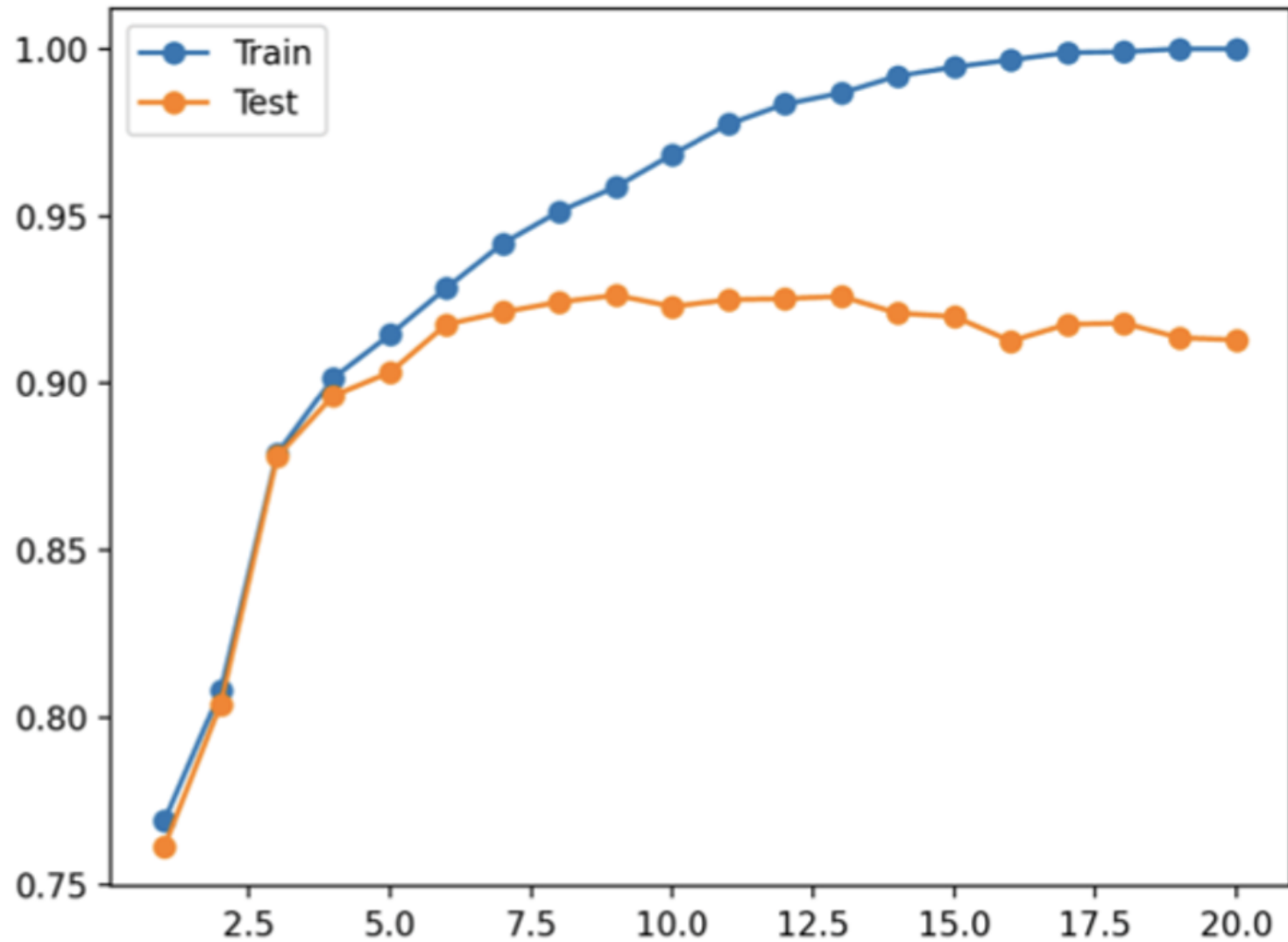


Overfitted

Overfitting e Underfitting



Overfitting



Funções de Ativação

Imagine cada "neurônio" como um filtro de informação. A função de ativação é a regra desse filtro, decidindo o que é importante passar adiante para a próxima etapa do aprendizado.

Um Filtro Inteligente:

- **Avalia a Informação:** Decide se um dado é relevante para o resultado final.
- **Aprendizado Detalhado:** Permite identificar padrões complexos, não apenas relações simples.
- **Impacto no Resultado:** A escolha do filtro certo é crucial para a precisão do seu projeto de IA.

Função	Ícone Representativo	Ideal Para (Quando usar)
ReLU	Uma rampa simples. Só passam valores positivos.	Camadas Internas: Eficiente para aprender padrões complexos. Ativa se o valor é positivo.
Sigmoid	Uma curva suave entre 0 e 1.	Resultado Binário: Útil na camada final para respostas de "sim" ou "não".
Linear	Uma linha reta.	Predições na Camada Final: Usada quando o objetivo é prever um valor contínuo (ex: preço, temperatura).
Softmax	Várias setas se normalizando em proporções.	Múltiplas Categorias: Essencial na camada final para classificar entre diversas opções.
Tanh	Uma curva "S" entre -1 e 1.	Alternativa para Camadas Internas: Pode ser útil com dados centrados em zero.

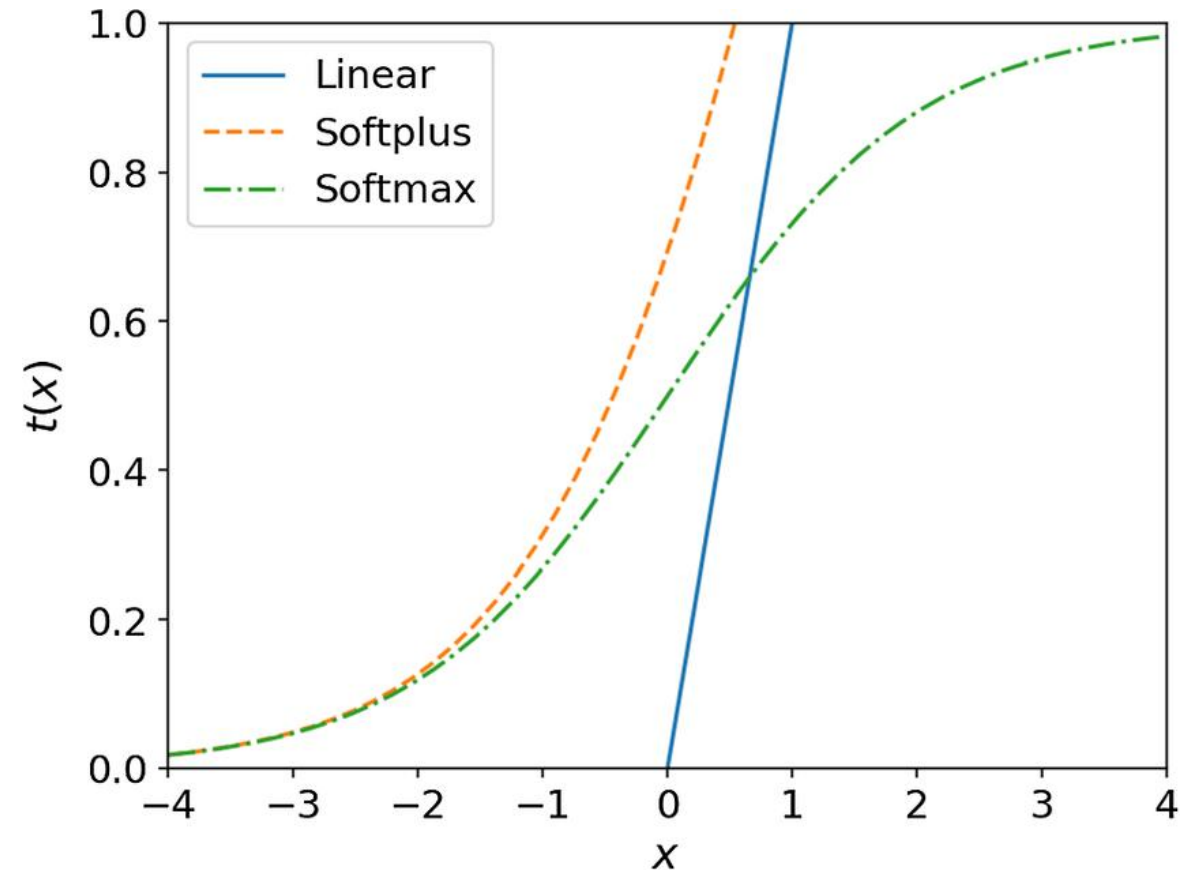
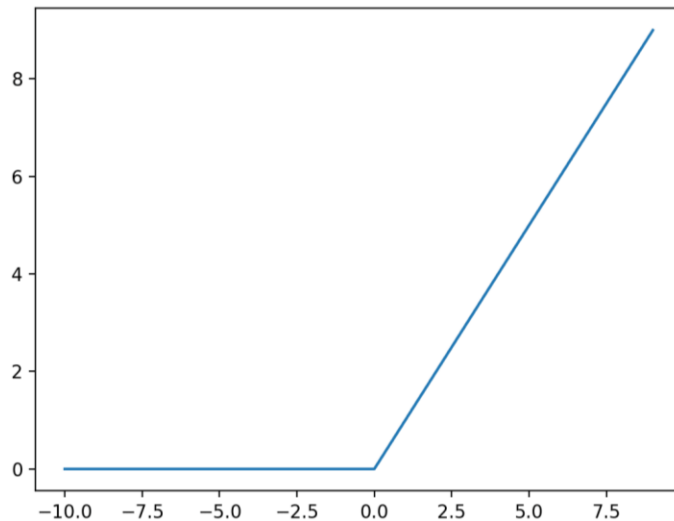
Funções de Ativação - Aplicações Práticas das Funções

Veja como as funções de ativação atuam em diferentes cenários:

- **Previsão de Cancelamento:** (Cliente vai cancelar? Sim/Não)
 - Use **Sigmoid** na camada final para obter a probabilidade de cancelamento.
- **Classificação de Conteúdo:** (Notícia é sobre Política, Esporte ou Economia?)
 - Use **Softmax** na camada final para ver a probabilidade de cada categoria.
- **Reconhecimento de Padrões em Imagens:** (Identificar objetos em uma foto)
 - Use **ReLU** nas camadas intermediárias para que a rede aprenda as características visuais.
- **Previsão de Vendas:** (Qual será o valor de vendas no próximo mês?)
 - Use **Linear** na camada final para obter uma previsão numérica direta.

Funções de Ativação - Pontos Chave

- **O que fazem?**
 - Decidem se a informação passa adiante.
- **Por que são importantes?**
 - Influenciam o que a rede consegue aprender.
- **Dicas:**
 - **Interno da rede:** Geralmente ReLU.
 - **Resultado Sim/Não:** Use Sigmoid.
 - **Resultado em Categorias:** Use Softmax.
 - **Resultado Numérico (Predição):** Use Linear.



Otimizadores – O GPS da Rede Neural

Se a função de ativação é o *motor de decisão* do neurônio, o **Otimizador** é o **GPS que guia o aprendizado** da rede inteira. Sua missão é encontrar o caminho mais rápido e eficiente para o melhor resultado, ajustando os parâmetros da rede para minimizar os erros.

Otimizador	Estratégia de Aprendizado	Quando usar?
SGD	O Caminhante Metódico: Simples e confiável. Dá um passo de cada vez na direção do menor erro. Pode ser mais lento.	Bom para datasets muito grandes e como base de aprendizado.
RMSprop	O Piloto de Rally: Adapta-se bem a "terrenos" (dados) irregulares e barulhentos, ajustando a velocidade para cada obstáculo.	Eficiente em problemas mais complexos e com sinais de erro muito variáveis.
Adam	O Piloto Inteligente (O Mais Popular): Combina a velocidade com a capacidade de se adaptar às curvas do problema. É rápido, eficiente e robusto.	A escolha padrão para a maioria dos projetos. É o otimizador mais recomendado para começar e geralmente oferece os melhores resultados.

Dicas – Escolhendo o modelo mais apropriado

A escolha entre modelos lineares (como regressão) e redes neurais depende da complexidade do problema.

Tipo de Problema	Modelo Sugerido	Justificativa
Previsão Simples (Relações Lineares): Prever o preço de uma casa com base no tamanho.	Regressão Linear	Simples, fácil de interpretar e eficiente para relações diretas.
Classificação Binária Simples: Prever se um cliente vai comprar (Sim/Não) com base em poucos dados.	Regressão Logística	Adequada para problemas com duas classes, fornece probabilidades e é bem compreendida.
Padrões Complexos (Não Lineares): Reconhecimento de imagens, processamento de linguagem natural, previsões com muitas variáveis e interações.	Redes Neurais	Capazes de aprender relações complexas e não lineares nos dados, adaptando-se a padrões sutis.

Dicas – Ajustando a "Velocidade" do Aprendizado

O **Learning Rate** é como o tamanho dos "passos" que o otimizador dá ao tentar encontrar a melhor configuração da rede.

- **Learning Rate Muito Alto:** Os passos são muito grandes, e a rede pode "pular" a solução ideal, oscilando e não convergindo.
- **Learning Rate Muito Baixo:** Os passos são muito pequenos, e o aprendizado pode ser muito lento, levando muito tempo para encontrar uma boa solução ou ficando preso em mínimos locais (soluções não ótimas).

Dicas:

- **Comece com valores pequenos:** Experimente valores como 0.01, 0.001 ou 0.0001.
- **Observe a função de perda (Loss):** Se a perda diminui muito lentamente, pode ser necessário aumentar o learning rate. Se a perda oscilar muito ou aumentar, pode ser necessário diminuir.
- **Learning Rate Schedules:** Algumas técnicas ajustam o learning rate ao longo do treinamento, começando alto para uma convergência rápida e diminuindo para um ajuste fino.

Dicas – Arquitetura da Rede: Camadas e Neurônios

Definir o número de camadas e neurônios por camada é um desafio empírico. Não há uma fórmula única, mas algumas heurísticas podem ajudar:

- Poucas Camadas e Neurônios:** Pode ser insuficiente para aprender padrões complexos, levando a *underfitting* (o modelo não se ajusta bem aos dados).
- Muitas Camadas e Neurônios:** Pode levar a *overfitting* (o modelo aprende demais os ruídos dos dados de treinamento e não generaliza bem para dados novos). Também aumenta o tempo de treinamento.

Dicas:

- Comece simples:** Para problemas iniciais, uma ou duas camadas ocultas podem ser suficientes.
- Considere a complexidade dos dados:** Dados mais complexos geralmente exigem redes maiores.
- Validação Cruzada:** Use técnicas de validação para avaliar o desempenho do modelo em dados não vistos e identificar overfitting.
- Experimentação:** Teste diferentes arquiteturas e observe o desempenho.
- Transfer Learning:** Para problemas com poucos dados, utilize modelos pré-treinados em tarefas semelhantes.